

ANA RAQUEL FERNANDES CUNHA

POSTECH

IA PARA DEVS

IAs GENERATIVAS

AULA 02

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	44
REFERÊNCIAS.....	45

EMSE

O QUE VEM POR AÍ?

A natureza inspirou muitas criações para os seres humanos, tais como as aves nos inspiraram a voar com os aviões, a planta bardana inspirou a criação do velcro e o cérebro humano inspirou a criação das chamadas “Redes Neurais Artificiais” (RNA), consideradas como **aprendizado profundo**. As redes neurais artificiais são poderosas e escaláveis, sendo muito utilizadas para lidar com **grandes tarefas altamente complexas do aprendizado de máquina**, tais como classificação de imagens, reconhecimento de fala, criação de imagens, chats poderosos e até mesmo o aprendizado de jogar videogames.

Para aprender sobre IA generativa, é preciso conhecer muito bem os fundamentos de **deep learning** e **processamento de linguagem natural**. Conhecer os fundamentos possibilita compreender como os modelos generativos são criados e como chegamos nessa evolução. Nesta aula, você vai aprender sobre as principais técnicas que possibilitaram a evolução desses modelos. Vamos lá?

HANDS ON

Que tal aprender na prática como funciona um modelo de deep learning? Nessas aulas, você vai aprender na prática com o python como funcionam os **embeddings** e as **deep learnings**.

EMEND

SAIBA MAIS

Conhecendo mais sobre deep learning: o que são os neurônios

Para entendermos como funcionam as redes neurais artificiais, vamos entender primeiro como funciona um neurônio biológico. O neurônio é uma célula encontrada principalmente no córtex cerebral, composto por um corpo celular contendo o **núcleo** e a maioria dos componentes complexos da célula e muitas extensões de ramificação chamadas de **dendritos**, além de uma extensão muito longa chamada **axônio**.

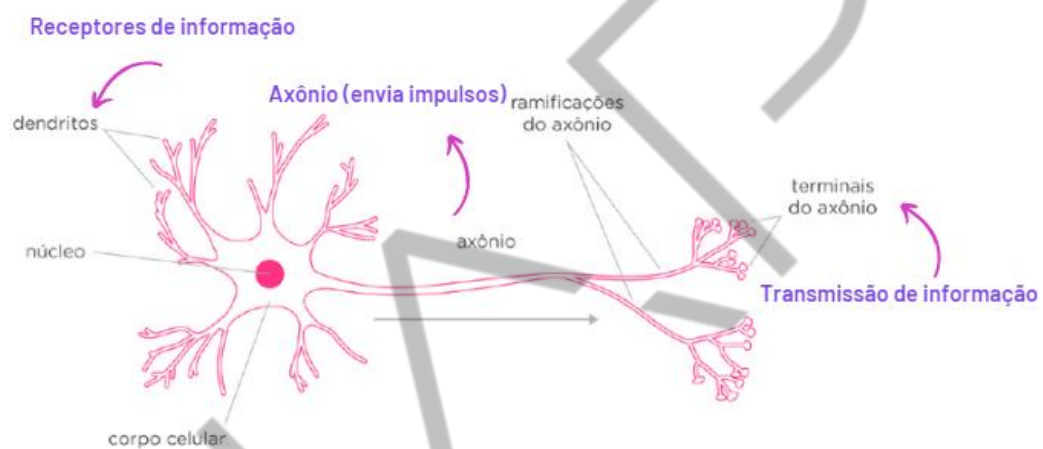


Figura 1 - Neurônio biológico
Fonte: elaborado pela autora (2024)

Perto de sua extremidade, o axônio divide-se em muitos ramos chamados de **telodendros** e na ponta desses ramos estão estruturas minúsculas chamadas **terminais sinápticos** (ou **sinapses**) que são conectados aos dendritos de outros neurônios.

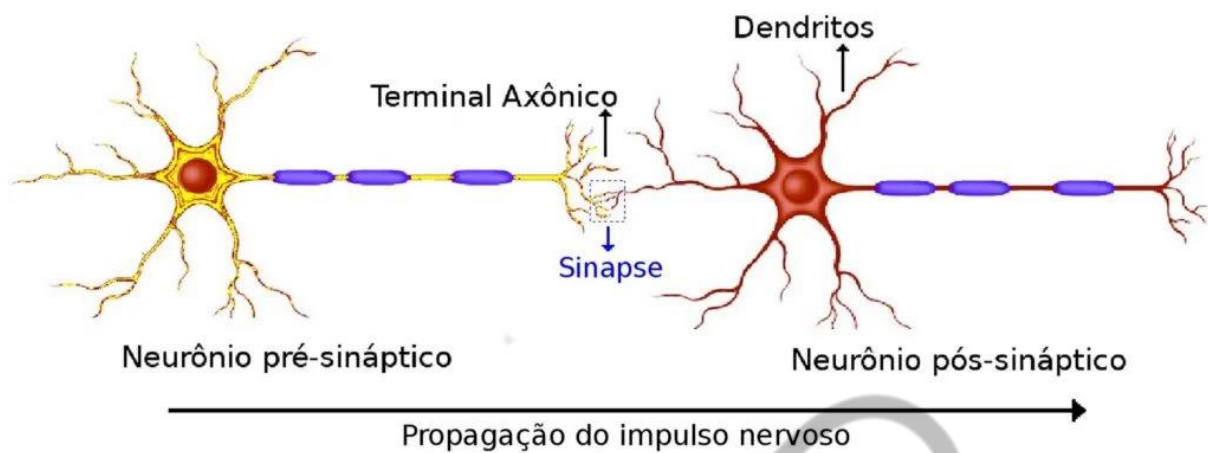


Figura 2 - Sinapse – Comunicação entre neurônios
Fonte: <https://escolaeducacao.com.br/sinapses/> (2024)

Os neurônios biológicos recebem curtos impulsos elétricos de outros neurônios por meio dessas sinapses chamadas sinais. É por meio das sinapses que os neurônios transmitem as informações de um para outro. Agora, imagine uma rede vasta de bilhões de neurônios conectados a milhares de outros neurônios? Essa é a ideia da rede neural!

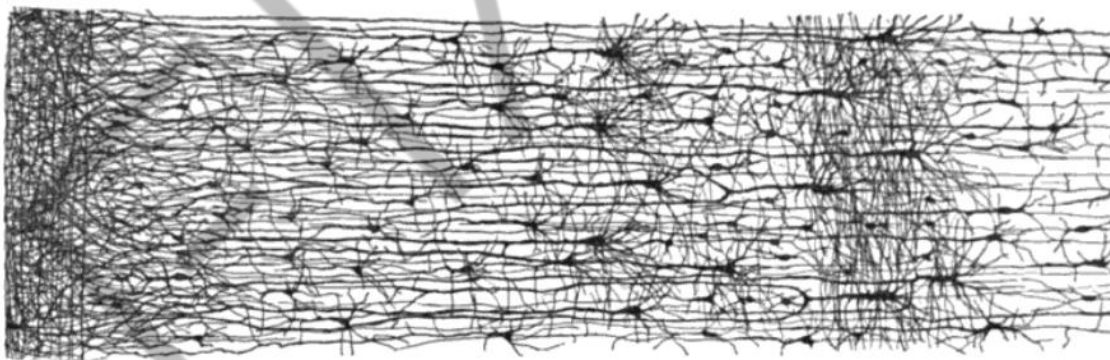


Figura 3 - Múltiplas camadas em uma rede neural biológica (córtex humano)
Fonte: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow [s.d]

Com base no comportamento do neurônio, McCulloch e Pitts (pioneiros na área) propuseram um modelo simples conhecido como **neurônio artificial**, possuindo uma ou mais entradas binárias (ligado/desligado).

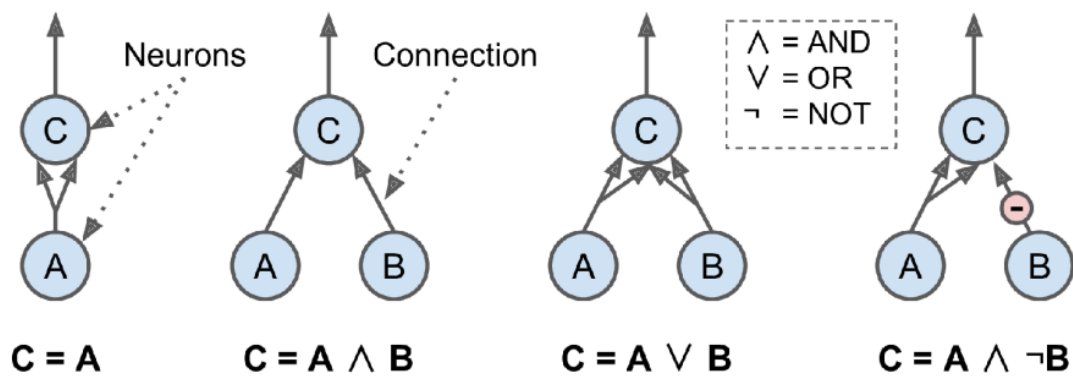


Figura 4 - Representação do funcionamento do neurônio artificial com lógica simples
 Fonte: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow [s.d]

Logo em 1957, inspirado nos trabalhos anteriores, Frank Rosenblatt inventou o modelo de **neurônio Perceptron**, uma arquitetura bem simples de RNA:

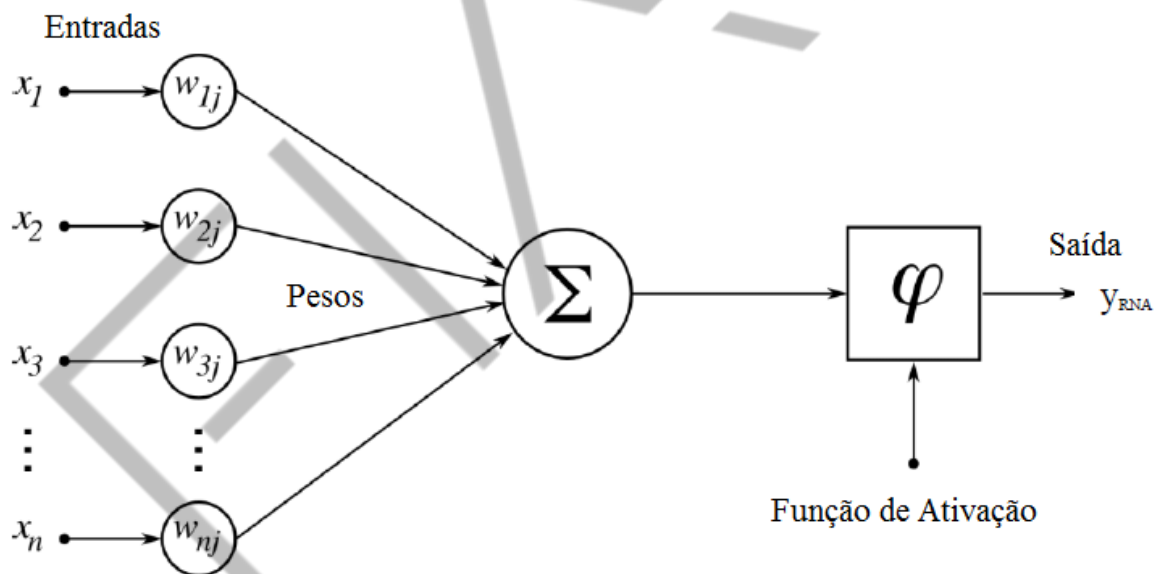


Figura 5 - Representação do neurônio artificial
 Fonte: https://www.researchgate.net/figure/Figura-1-Representacao-do-neuronio-artificial_fig1_329245206 (2016)

Conhecendo os modelos de deep learning

Multilayer Perceptron (Redes Neurais Multicamadas)

A famosa rede multicamadas de neurônios é basicamente uma evolução do que Frank Rosenblatt inventou com o modelo de perceptron. Em vez da rede ser

composta por apenas uma única camada de neurônio, ela possui várias camadas. O treinamento dessa rede consiste basicamente em modelar as correlações e padrões nos dados de acordo com uma target predestinada (estamos falando aqui de modelos supervisionados). O seu treinamento envolve o **ajuste dos parâmetros ou pesos e bias** do modelo para **minimizar o erro**.

Seu funcionamento consiste basicamente em o algoritmo alimentar cada instância de treinamento para a rede calcular a saída de cada neurônio em cada camada consecutiva. Em seguida, ele **mede o erro de saída da rede** (isto é, a diferença entre a saída desejada e a saída real da rede) e **calcula o quanto cada neurônio contribui para o erro** de saída na última camada oculta. Como próximo passo, o modelo passa a medir a quantidade dessas contribuições de erro provenientes de cada neurônio na camada oculta anterior, e assim por diante até o algoritmo alcançar a camada de entrada. Esta passagem reversa (também conhecida como **backpropagation**) mede a **eficiência do gradiente de erro em todos os pesos de conexão da rede**.

A grande ideia por trás da rede neural é encontrar **quais são os melhores pesos ajustados pelo backpropagation, que ajusta uma função de custo** dada pelo gradiente descendente (derivada da função de custo).

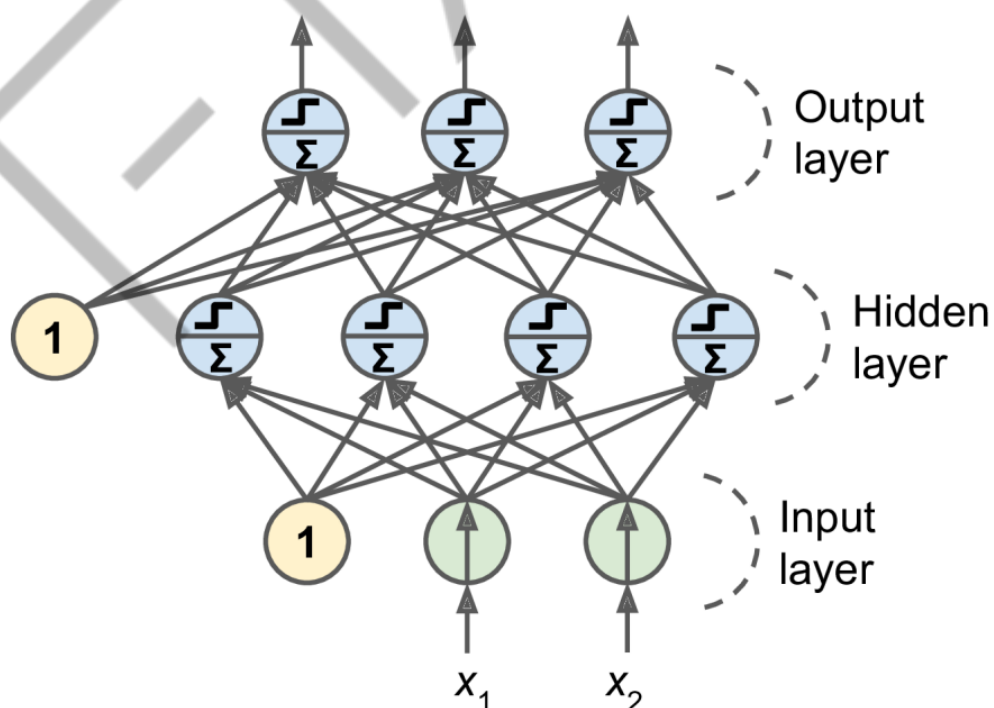


Figura 6 - Modelos de rede neural multicamadas
Fonte: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow [s.d]

Entendendo o modelo Backpropagation

O modelo de backpropagation é utilizado para fazer os ajustes dos pesos e de bias dos neurônios em relação ao erro. As redes neurais multicamadas basicamente funcionam utilizando a propagação e a retropropagação.

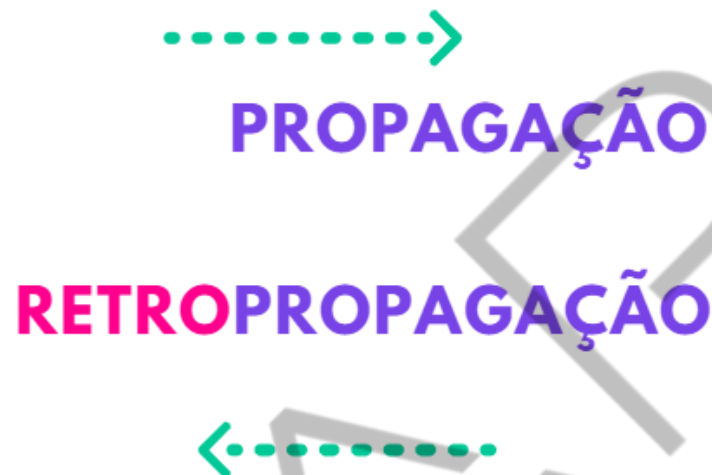


Figura 7 - Modelo Backpropagation
Fonte: Elaborado pela autora (2024)

Para você compreender melhor esse modelo, acompanhe as próximas figuras. Imagine que teremos a entrada das variáveis x_1 e x_2 na primeira camada da rede, a **Input Layer**:

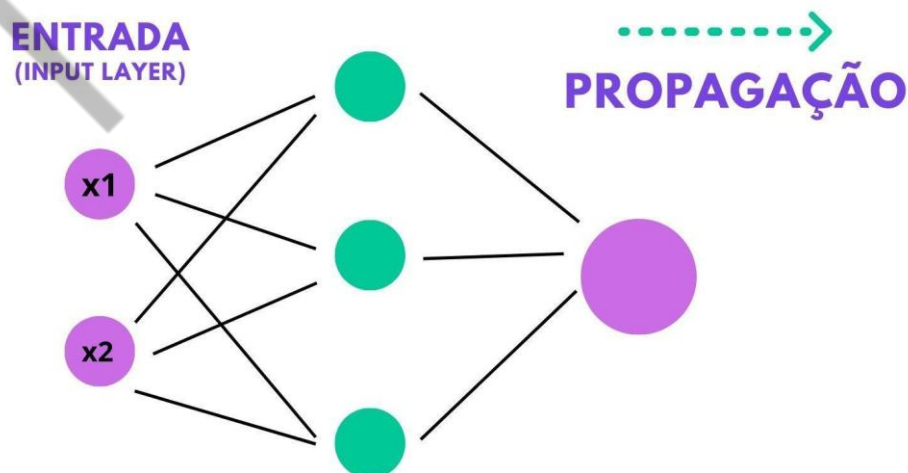


Figura 8 - Explicação Backpropagation: camada de entrada
Fonte: elaborado pela autora (2024)

Ainda na camada de entrada, essas variáveis terão pesos atribuídos de forma aleatória a cada uma delas, para assim o neurônio ganhar força e ir para a próxima camada:

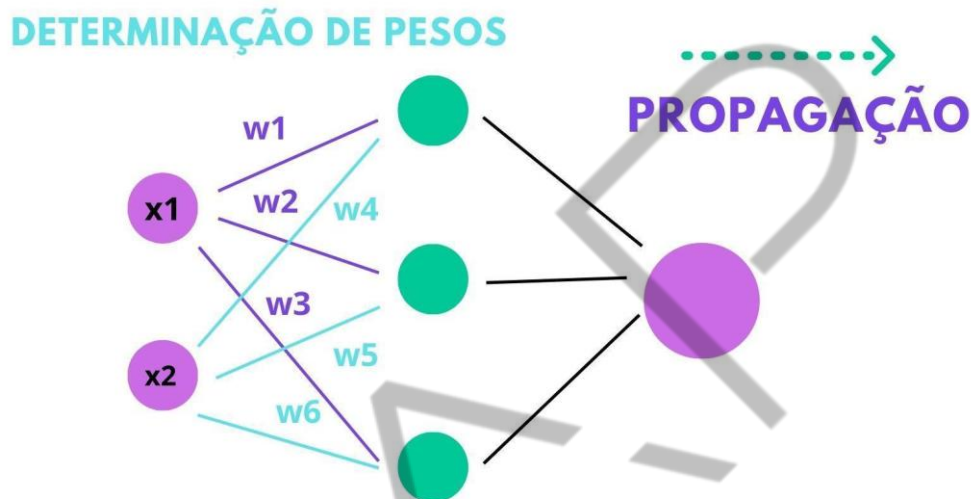


Figura 9 - Explicação Backpropagation: atribuição de pesos
Fonte: elaborado pela autora (2024)

Essas entradas vão passar pela camada conhecida como “Hidden Layer” (camada oculta). Nesse exemplo, mostramos apenas uma camada para simplificar a explicação, porém imagine aqui que podemos ter muitas camadas ocultas dentro de uma rede neural multicamadas. Em cada camada oculta, o cálculo do somatório (incluindo variáveis e pesos) é realizado e aplicado sobre uma função de ativação

(entraremos em mais detalhes sobre as funções de ativação mais adiante).

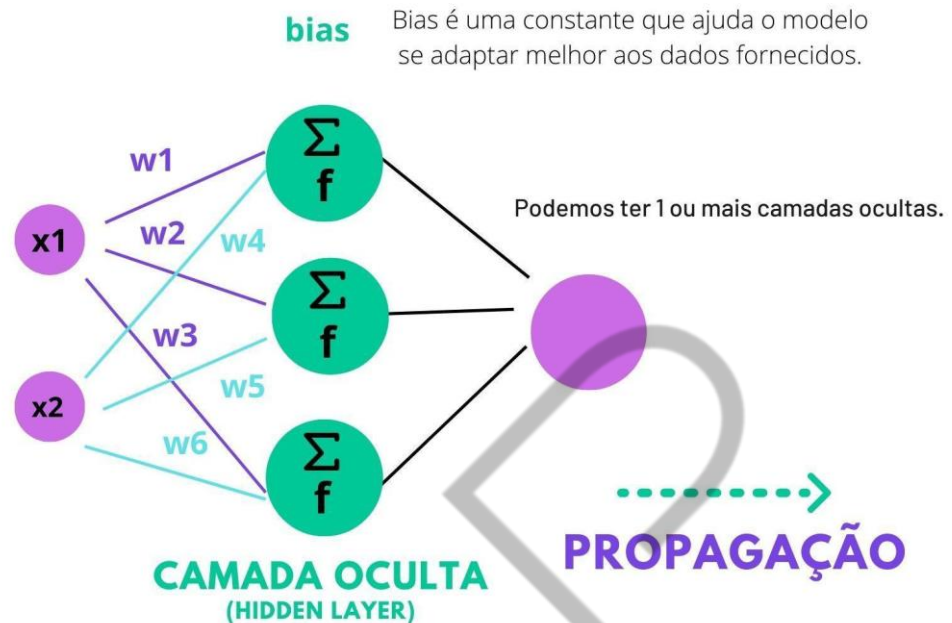


Figura 10 - Explicação Backpropagation: camada oculta
Fonte: elaborado pela autora (2024)

Depois de todos os cálculos serem realizados por todas as camadas ocultas, vamos ter uma saída da rede, a chamada “Output Layer” (camada de saída). Essa saída retornará um valor de erro dado a comparação do valor estimado versus o valor real.

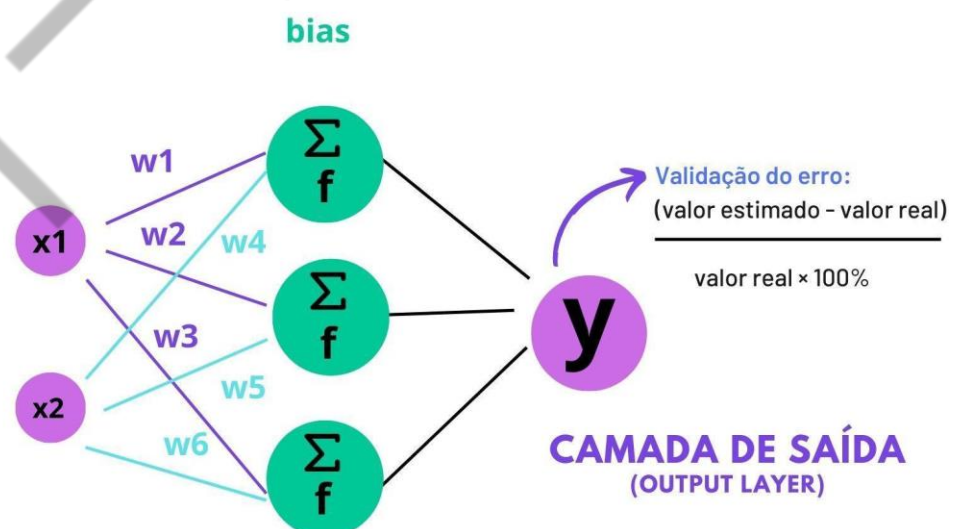


Figura 11 - Explicação Backpropagation: camada de saída
Fonte: Elaborado pela autora (2024)

Agora é a hora do diferencial das redes multicamadas: o backpropagation! Com o erro calculado na primeira passagem da rede, o backpropagation é responsável por retornar ao processo da rede desde o início das camadas e reprocessar todos os dados novamente, porém com pesos diferentes. Essa etapa é responsável por atualizar os pesos da rede e gerar um novo valor de saída. A ideia aqui é minimizar cada vez mais os erros gerados na camada de saída.

AJUSTE DE PESOS DA REDE

Com base no valor de resultado do erro, vamos ter um parâmetro de quanto a nossa rede acertou o resultado.

Nem sempre esses erros vão ter valores bons. É nesse caso aonde entra a nossa retro propagação da rede para ajustar os pesos da rede neural e encontrar um valor ideal para nosso resultado de saída.

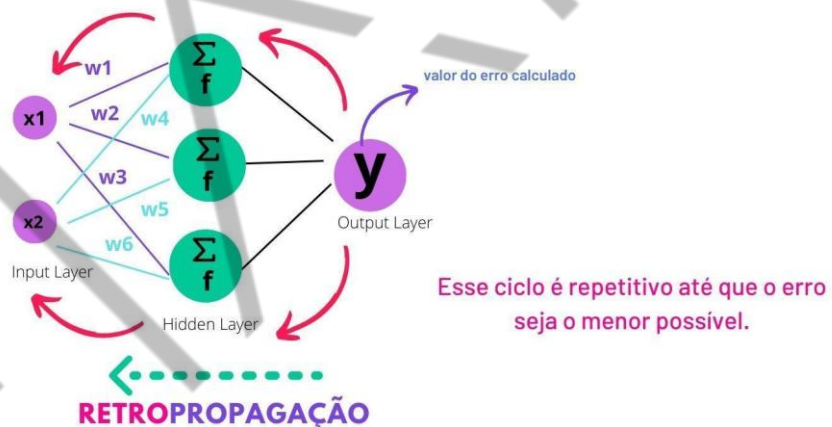


Figura 12 - Explicação Backpropagation: funcionamento
Fonte: elaborado pela autora (2024)

Mas você deve estar se perguntando, mas até quando esses pesos devem ser atualizados? Até quando a rede vai ficar nesse ciclo de vai e vem? O hiperparâmetro responsável por esse processo são as chamadas **epochs**. Podemos definir as epochs (épocas de processamento) como o **total de vezes que são executadas as redes neurais**. (Número máximo de épocas).

Os hiperparâmetros das redes neurais multicamadas

Podemos listar muitos hiperparâmetros a serem configurados em uma rede neural multicamadas, tais como:

Número de camadas ocultas: nem sempre apenas uma única camada de neurônio na rede será suficiente. As redes multicamadas possuem uma arquitetura composta por **várias camadas ocultas**, essas hierarquias de neurônios auxiliam a rede a **convergir mais rapidamente** para uma boa solução, também ajudando a melhorar a capacidade de **generalizar** para novos conjuntos de dados.

Para uma boa partida, inicie sua construção de redes multicamadas com uma ou duas camadas ocultas de neurônios (e, se for necessário, você vai construindo mais camadas para solucionar o seu problema).

Normalmente problemas muito complexos necessitam de mais camadas ocultas. Para problemas complexos você pode aumentar gradualmente o número de camadas até começar a sobreajustar o conjunto de treinamento. Tarefas muito complexas como classificação de imagens ou reconhecimento de imagens necessitam de mais dezenas de camadas ocultas para tal complexidade.

Número de neurônios dentro das camadas: o número de neurônios nas camadas de entrada e saída é determinado pelo tipo de entrada e saída que sua tarefa requer. Assim como para o número de camadas, você pode tentar aumentar o número de neurônios gradualmente até que a rede comece a se sobreajustar. Em geral, você obterá mais vantagem aumentando o número de camadas do que o número de neurônios por camada.

Funções de ativações: o objetivo de uma função de ativação é **realizar a transformação não linear dos neurônios**. Aliás, uma rede neural sem a função de ativação é essencialmente um modelo de regressão linear. A função de ativação permite que as mudanças realizadas nos pesos e bias **causem uma alteração na saída final do modelo** (output).

$$Y = \text{Activation}(\Sigma(\text{weight} * \text{input}) + \text{bias})$$

Figura 13 - Função de ativação
 Fonte: <https://www.deeplearningbook.com.br/funcao-de-ativacao/> (2022)

Existem diversos tipos de funções de ativações, tais como:

Sigmóide: essa função de ativação tenta empurrar os valores de Y para os extremos. Esta é uma qualidade muito desejável quando tentamos classificar os valores para uma classe específica.

- Saídas com 0 ou 1 (retorna dados binários).
- Possui uma suavização.
- Função não linear.
- A função essencialmente tenta empurrar os valores de Y para os extremos.
- Trabalha apenas com valores positivos.
- O resultado da função pode ser uma combinação de diversas variáveis.
- Funciona muito bem para classificar os valores para até duas classes específicas.
- Pode apresentar problemas quando os gradientes se tornam muito pequenos. Isso significa que o gradiente está se aproximando do zero e a rede não está realmente aprendendo.

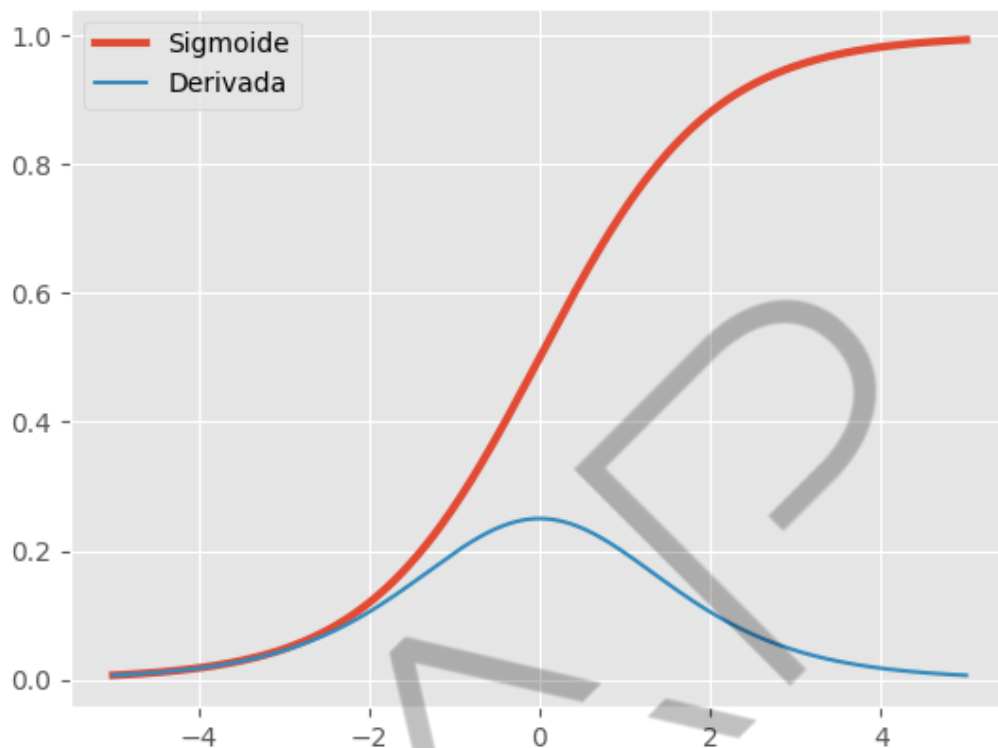


Figura 14 - Função de ativação sigmoide

Fonte: <https://matheusfacure.github.io/2017/07/12/activ-func/> (2017)

Tangente Hiperbólica (tanh): a função tangente hiperbólica basicamente funciona parecido com a sigmoide, porém varia de -1 a 1.

- A função tanh é uma versão escalonada da função sigmoide.
- Varia de -1 a 1.
- Função não linear.
- A TanH se aproxima mais da identidade, sendo assim uma alternativa mais atraente do que a sigmoide para servir de ativação às camadas ocultas das RNAs.

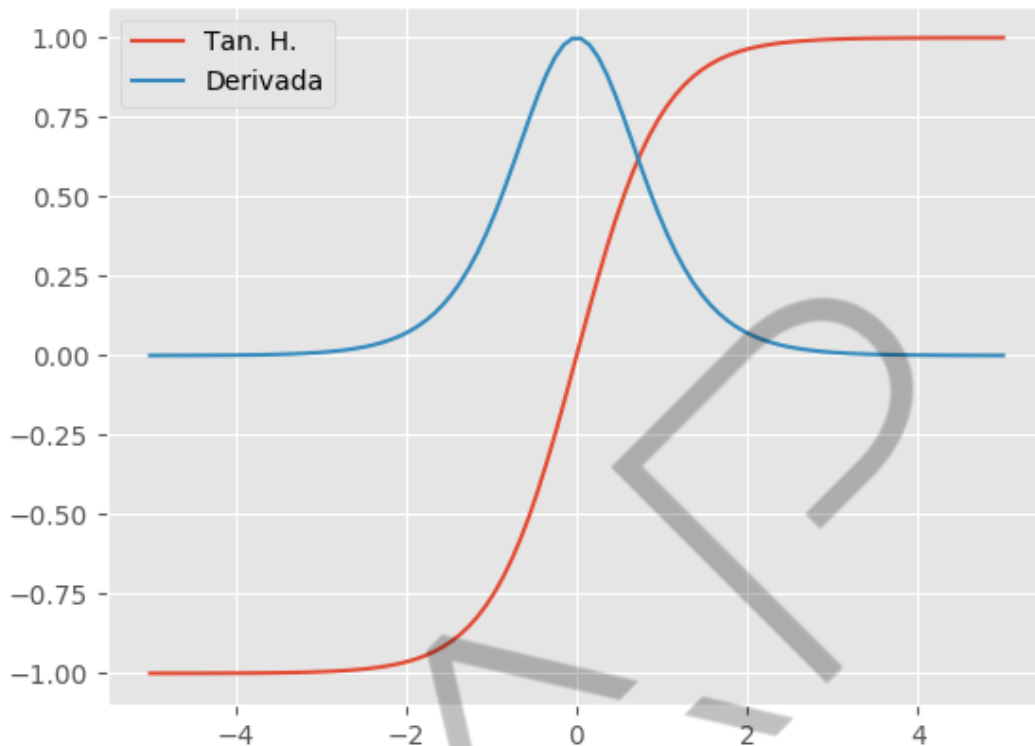


Figura 15 - Função de ativação tanh
 Fonte: <https://matheusfacure.github.io/2017/07/12/activ-func/> (2017)

ReLU (unidade linear retificada): a função ReLU é uma das mais utilizadas em redes neurais, pois uma das suas principais funções é não ativar todos os neurônios ao mesmo tempo (isso traz mais performance para a rede).

- A função ReLU é não linear.
- Não ativa todos os neurônios ao mesmo tempo. Se você olhar para a função ReLU e a entrada for negativa, ela será convertida em zero e o neurônio não será ativado.
- Menor complexidade computacional.
- ReLU também pode ter problemas com os gradientes que se deslocam em direção a zero.
- A função ReLU é uma função de ativação geral e é usada na maioria dos casos atualmente.

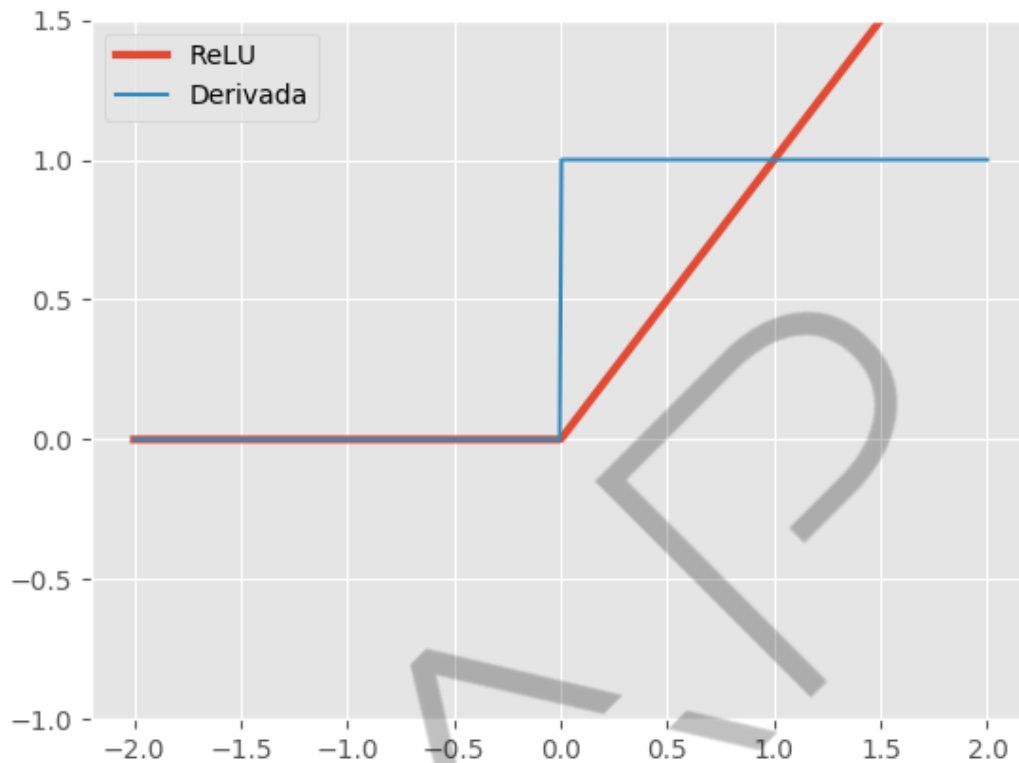


Figura 16 - Função de ativação ReLU
 Fonte: <https://matheusfacure.github.io/2017/07/12/activ-func/> (2017)

Leaky ReLU: a função Leaky ReLU é uma versão melhorada da ReLU, com a diferença de que valores menores que 0 recebem um componente linear de ativação.

- Versão melhorada da ReLU.
- Faz a ativação para valores $x < 0$.
- Para valores menores que 0, é definido um pequeno componente linear (0,01).
- A principal vantagem de substituir a linha horizontal é remover o gradiente zero.

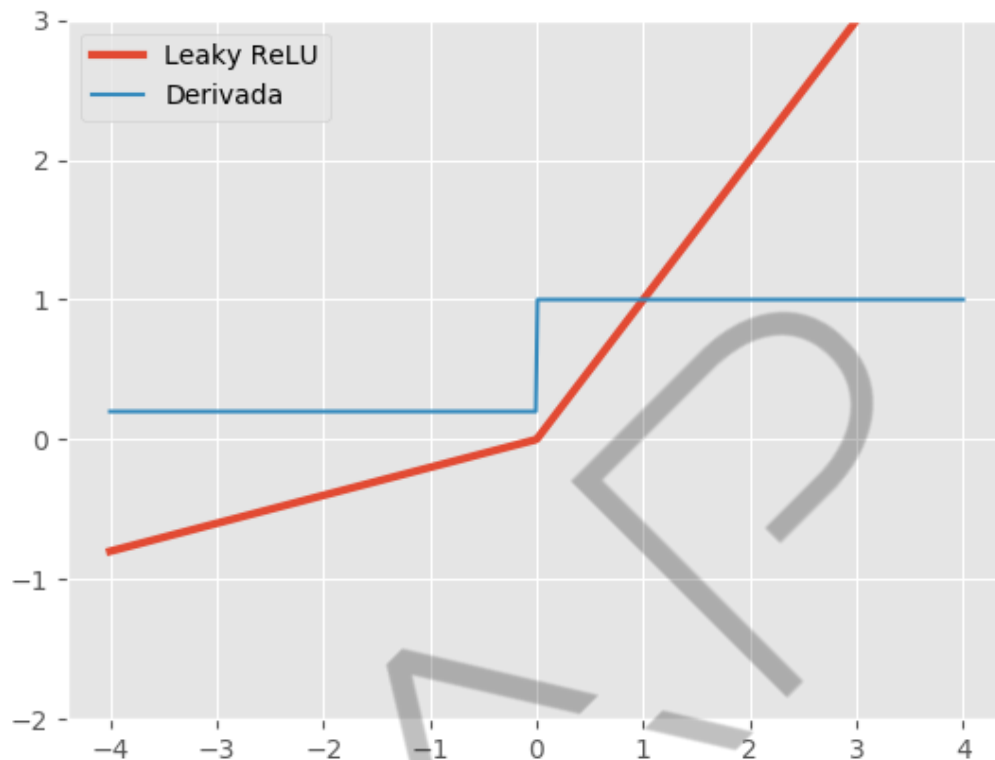


Figura 17 - Função de ativação Leaky ReLU
 Fonte: <https://matheusfacure.github.io/2017/07/12/activ-func/> (2017)

Softmax: a função Softmax é muito útil quando queremos lidar com problemas de multiclases. A função softmax transforma as saídas para cada classe em valores entre 0 e 1 e também divide pela soma das saídas.

- Também é um tipo de função sigmóide.
- Além de transformar as saídas para 0 e 1 também divide pela soma das saídas.
- Atribui a probabilidade de resultado pertencer a uma determinada classe.
- A função softmax é idealmente usada na camada de saída do classificador, onde estamos tentando gerar as probabilidades para definir a classe de cada entrada.
- Ideal quando estamos tentando lidar com várias classes.



Figura 18 - Função de ativação Softmax

Fonte: <https://acervolima.com/compreendendo-as-funcoes-de-ativacao-em-profundidade-1/> [s.d.]

Falando da aplicação das funções de ativações em redes neurais multicamadas, na maioria dos casos você pode utilizar a função ReLU nas camadas ocultas pelo fato de ser mais rápida para calcular do que outras funções de ativações. Outra vantagem é que o gradiente descendente (função de custo), não fica tão preso em platôs, graças ao fato de que ela não satura para grandes valores de entrada. Quando temos classes exclusivas, a função de ativação Softmax é uma boa opção para tarefas de classificação na camada de saída da rede. Para casos onde temos apenas duas classes de saída, a função de ativação sigmóide pode ser uma boa opção.

Entendendo o gradiente descendente

Você observou que o tal “gradiente descendente” foi muito citado, mas afinal, o que é um gradiente descendente?

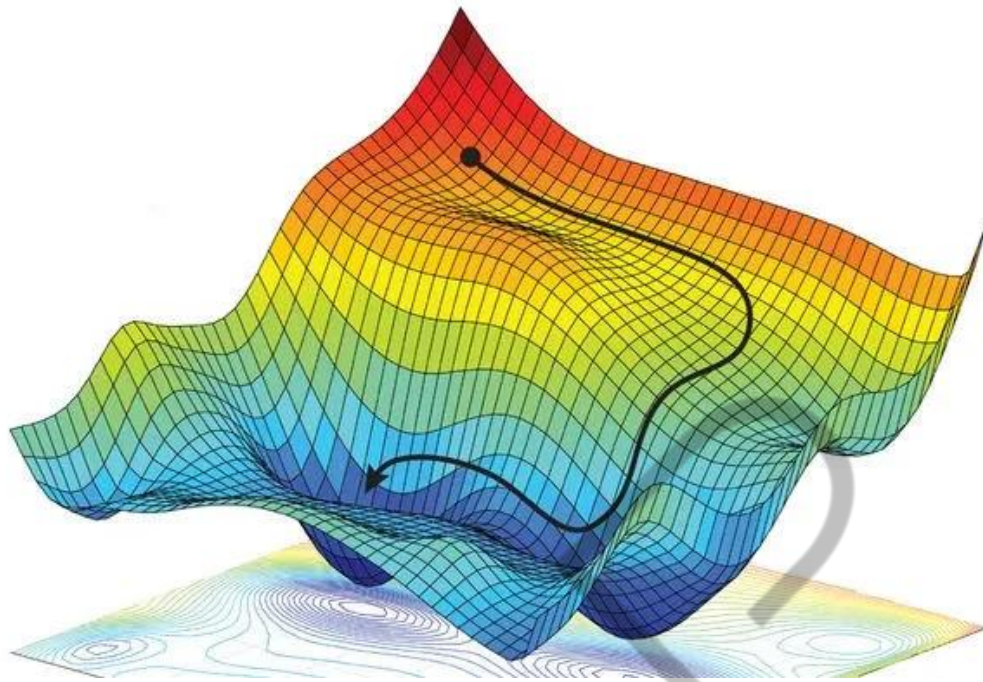


Figura 19 - Otimização de gradiente descendente

Fonte: <https://medium.com/data-hackers/gradientes-descendentes-na-pr%C3%A1tica-melhor-jeito-de-entender-740ef4ff6c43> (2020)

A descida do gradiente é um **algoritmo de otimização para encontrar os valores de parâmetros (pesos) de uma função que minimizam uma função de custo**. Você também pode pensar em um gradiente como a inclinação de uma função. Quanto maior o gradiente, mais inclinada é a subida e mais rápido o modelo pode aprender. Mas se a inclinação for zero, o modelo para de aprender. O Gradiente Descendente (GD) é um algoritmo utilizado para encontrar o **mínimo de uma função de forma iterativa**.

Pensando na questão da matemática por trás dos gradientes, podemos dizer que o gradiente descendente é a **derivada da função do erro em relação a nosso peso**.

Repita "N" vezes:

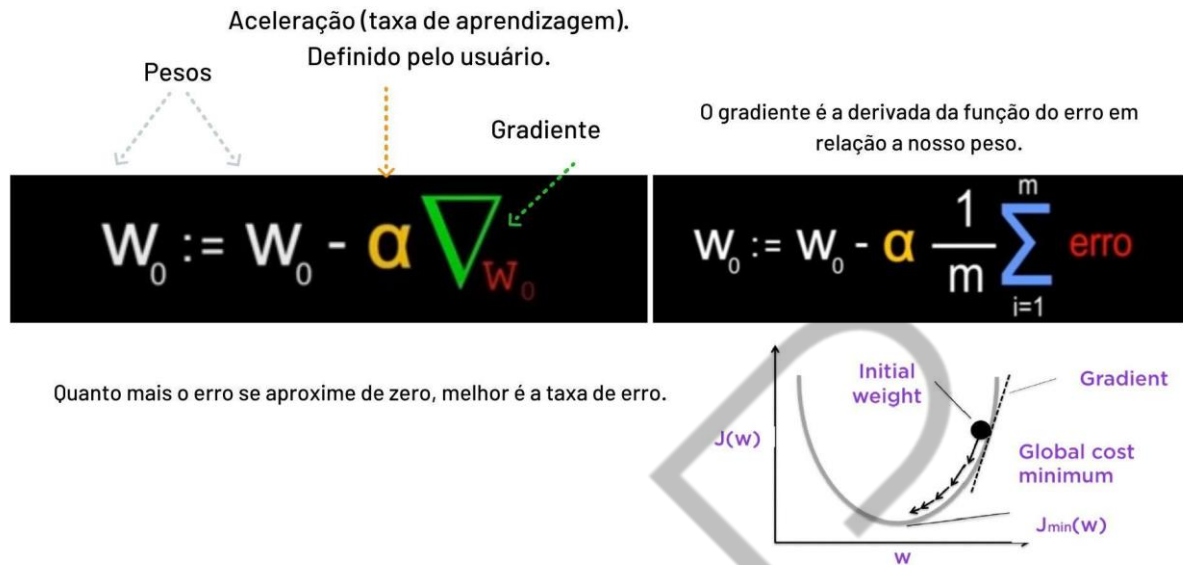
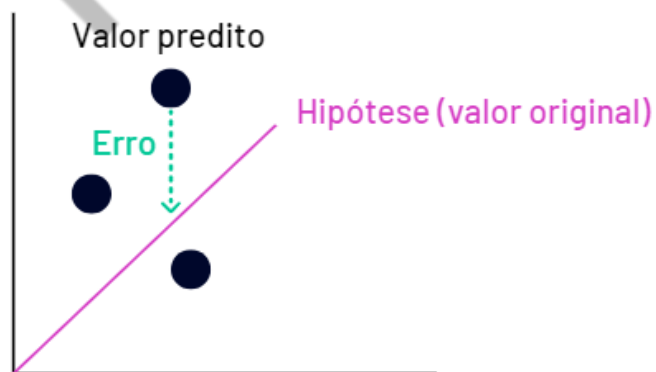


Figura 20 - Gradiente descendente (explicação matemática)
Fonte: Elaborado pela autora (2024)

Este processo é repetido até que o **custo dos coeficientes (função de custo)** seja 0,0 ou próximo o suficiente de zero, indicando que as saídas da rede estão cada vez mais próximas dos valores reais (saídas desejadas).



Taxa de aprendizagem (aceleração)
Gradiente (direção) Como procurar a
parte mais baixa da função de custo?

Figura 21 - Minimização da função de custo
Fonte: elaborado pela autora (2024)

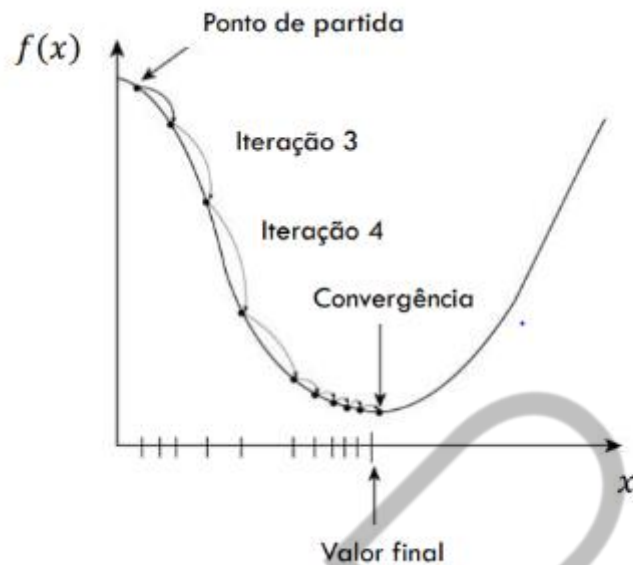
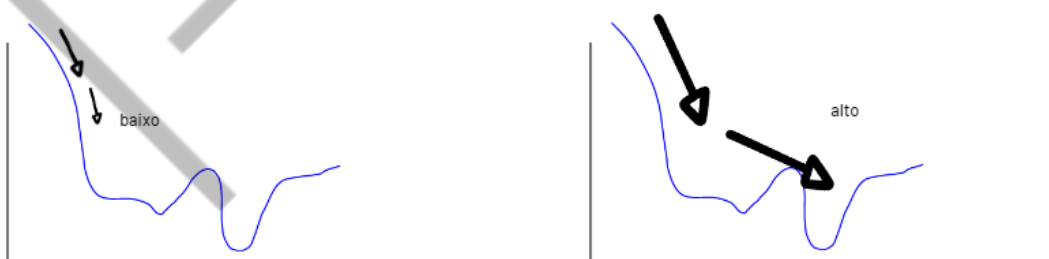


Figura 22 - Gradiente descendente
Fonte: elaborado pela autora (2024)

A performance que ocorre durante a descida do gradiente é controlada pela taxa de aprendizagem, também conhecida como **learning rate**. Podemos dizer que a learning rate controla o tamanho dos “passos” das interações para encontrar um melhor mínimo local (menor erro). Uma taxa de aprendizagem baixa pode ser mais precisa, porém, pode demorar para encontrar o mínimo local. Uma taxa de aprendizado alta pode deixar passar o melhor mínimo local.



Tente ... 0,001 – 0,003 – 0,01 – 0,03 – 0,1 – 0,3 –
1 ... Suba/Desça ~ 3x em 3x

Figura 23 - Learning rate e a iteração no gradiente
Fonte: elaborado pela autora (2024)

A learning rate é um hiperparâmetro muito importante a ser configurado nas redes neurais, e normalmente tentamos valores muito baixos quando buscamos uma maior eficiência da convergência do erro, casando, então, com um número maior de épocas de processamento, ou também podemos ter learning rate nem tão baixas assim e com épocas menores de processamento. Aqui a ideia é sempre testar qual é a melhor configuração e com qual delas o seu modelo convergirá melhor.

Entendendo as redes neurais convolucionais

Quando pensamos em IA generativa, a parte da criação de imagens está totalmente associada com as redes neurais convolucionais (CNN). Assim como aprendemos nas redes neurais multicamadas, as CNNs também são inspiradas em fatores biológicos. Em 1958 e 1959, David H. Hubel e Torte Wiesel realizaram uma série de experimentos com gatos, na qual proporcionaram o aprendizado sobre a estrutura do córtex visual.

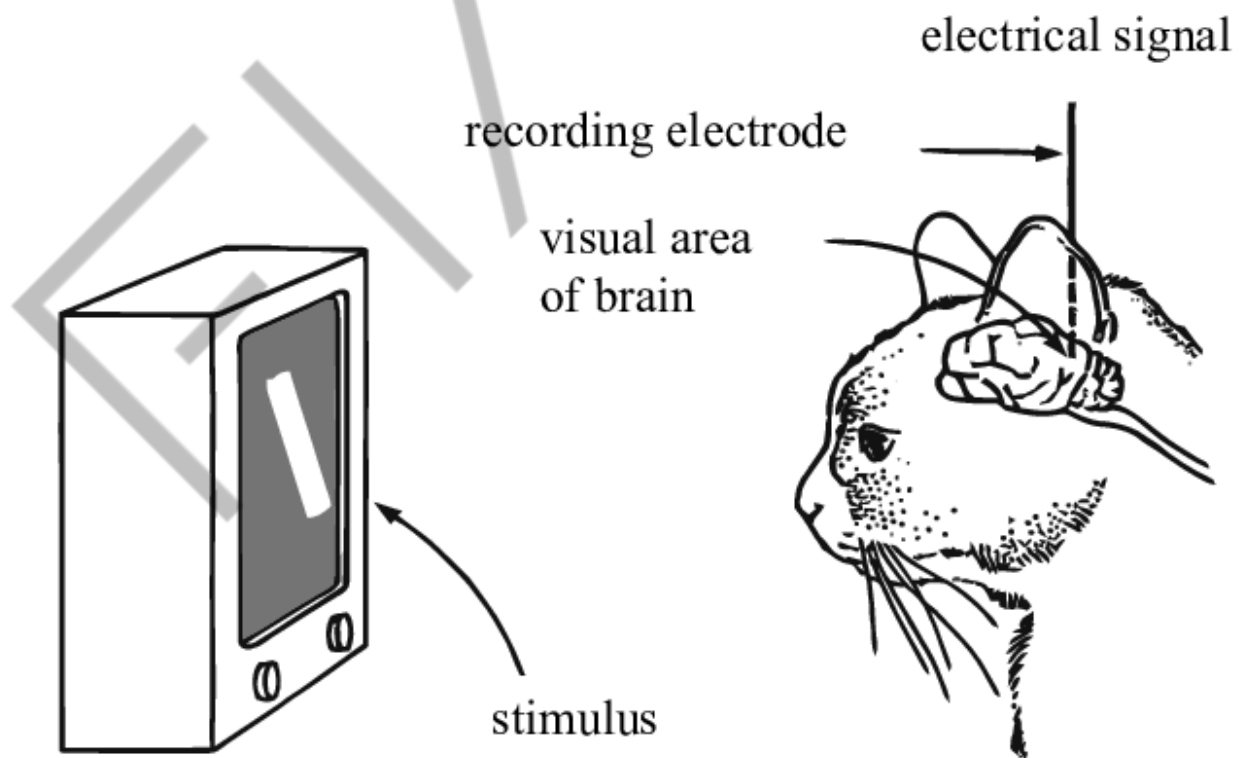


Figura 24 - Experimento realizado no córtex visual em um gato

Fonte: https://www.researchgate.net/figure/fig-In-the-classic-neuroscience-experiment-Hubel-and-Wiesel-discovered-a-cats-visual_fig1_335707980 (2019)

Hubel e Wiesel descobriram que os neurônios do córtex visual de um gato **são ativados juntos quando são expostos a algumas linhas ou curvas**. Inspirado no córtex visual, saber distinguir diferentes imagens é possível quando **identificamos e comparamos diferentes características em pontos específicos na imagem**. Eles também notaram que alguns neurônios têm campos receptivos maiores e reagem a padrões mais complexos (que são combinações dos padrões de nível inferior). Essas observações levaram à ideia de que os neurônios de nível superior são baseados nas saídas dos neurônios vizinhos de nível inferior. Esta poderosa arquitetura é capaz de detectar todos os tipos de padrões complexos em qualquer área do campo visual.

Como o computador identifica imagens

Você deve estar se perguntando como é que o computador consegue identificar uma imagem, certo? Sabemos que trabalhar com dados não estruturados pode ser uma tarefa não tão simples. As entradas das redes CNNs basicamente são imagens, o que torna o seu input de dados diferenciado. Sabemos que o computador entende números, correto? Pois bem, as redes neurais basicamente recebem uma **matriz de pixels da imagem como entrada da rede**.

Existem os tipos de imagens em escala de cinza (grayscale) e as imagens coloridas (rgb). Basicamente essas imagens são compostas por pixels que variam entre 0 a 255 mostrando a intensidade de cores. A imagem a seguir, por exemplo, está representada por uma matriz 2D, onde cada posição da imagem é composta por elementos que variam de 0 (preto) a 255 (branco). Quanto mais próximo de 255, mais clara é a cor em escala de cinza.

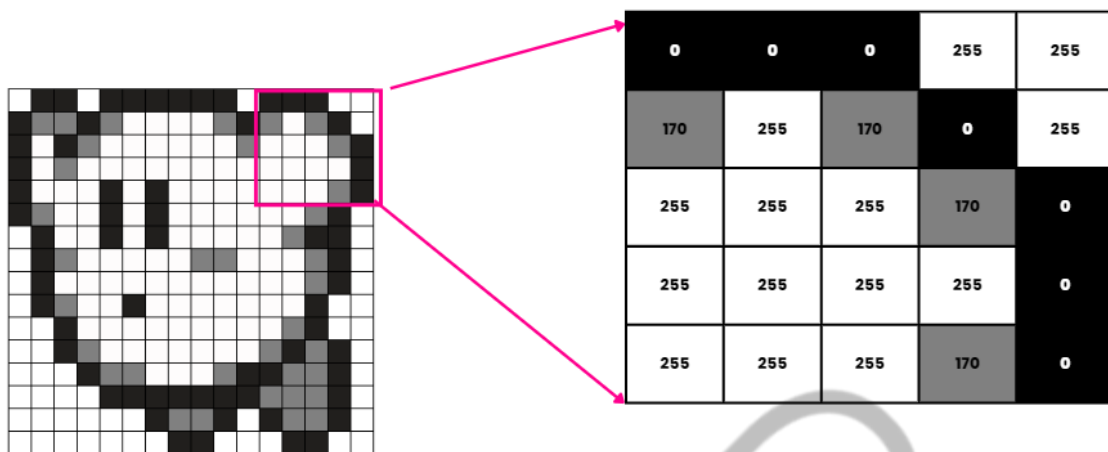


Figura 25 - Pixels de uma imagem grayscale
Fonte: elaborado pela autora (2024)

Já as imagens em escala colorida (rgb), são representadas por uma matriz 3D, combinando cores vermelho, verde e azul. O propósito principal do sistema RGB é a reprodução de cores em dispositivos eletrônicos como monitores de TV, computadores, mídias digitais e celulares em geral.

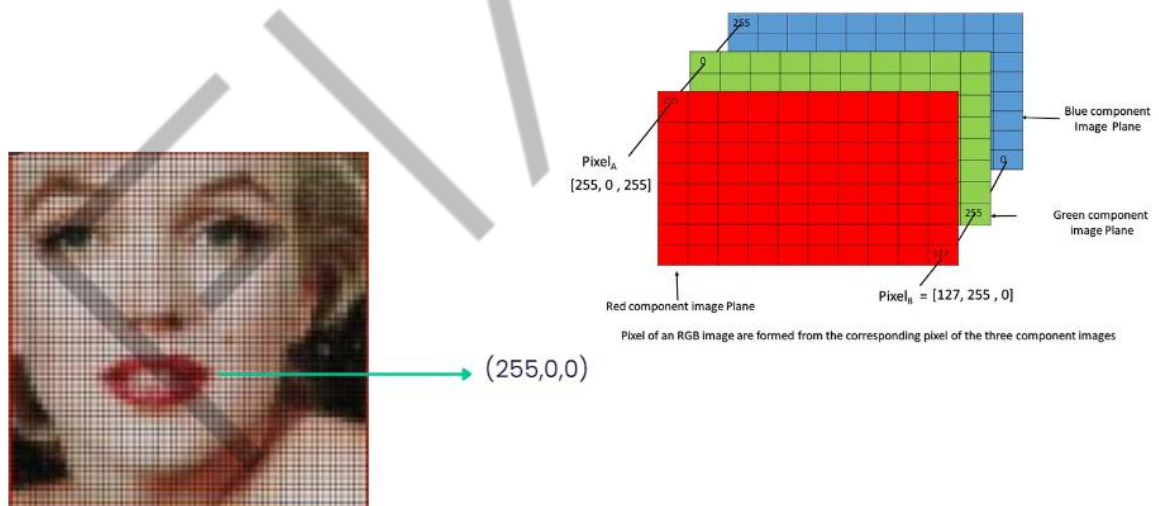


Figura 26 - Pixels de uma imagem rgb
Fonte: elaborado pela autora (2024)

Arquitetura das redes neurais convolucionais

Quando pensamos em redes CNN, a camada mais importante e que faz total diferença entre as redes neurais comuns é a **camada convolucional**. Os neurônios na primeira camada convolucional não estão conectados a cada pixel na imagem de

entrada, mas apenas a pixels em campos respectivos. E assim, cada neurônio da segunda camada convolucional está conectado apenas a neurônios localizados dentro de um pequeno retângulo na primeira camada. As camadas convolucionais são responsáveis pela etapa de aprendizagem das partes da imagem como um todo.

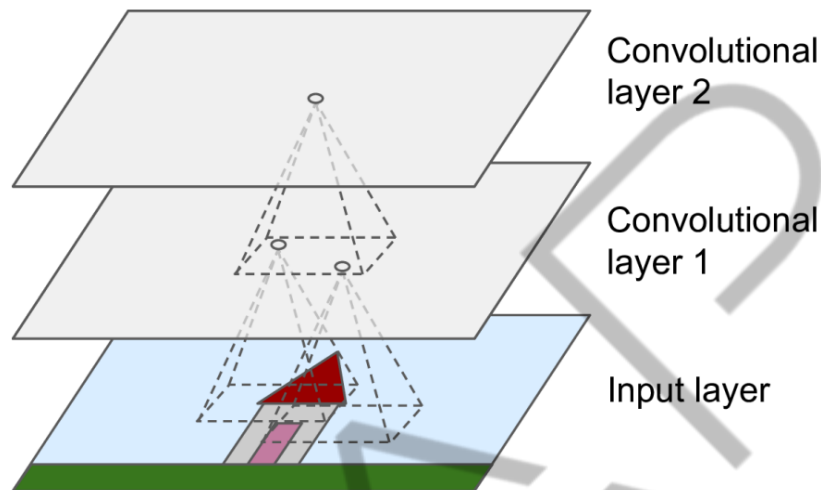


Figura 27 - Camadas NN com campos receptivos locais retangulares
Fonte: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow [s.d.]

Para você entender de forma simples o funcionamento das redes convolucionais, vamos utilizar um exemplo clássico nos estudos das redes neurais. Imagine que a rede neural convolucional será treinada para classificar dígitos de 0 a 9 escritos a mão. Você sabe que cada pessoa possui uma caligrafia única e nem sempre as escritas são iguais. Um grande desafio para a rede, certo?



Figura 28 - Amostra de dígitos escritos à mão

Fonte: <https://www.analyticsvidhya.com/blog/2021/06/a-beginners-guide-to-codeless-deep-learning-mnist-digit-classification/> (2021)

Vamos pegar um exemplo do dígito zero (0). Imagine que essa imagem será a entrada da rede e a arquitetura possui a estrutura a seguir:

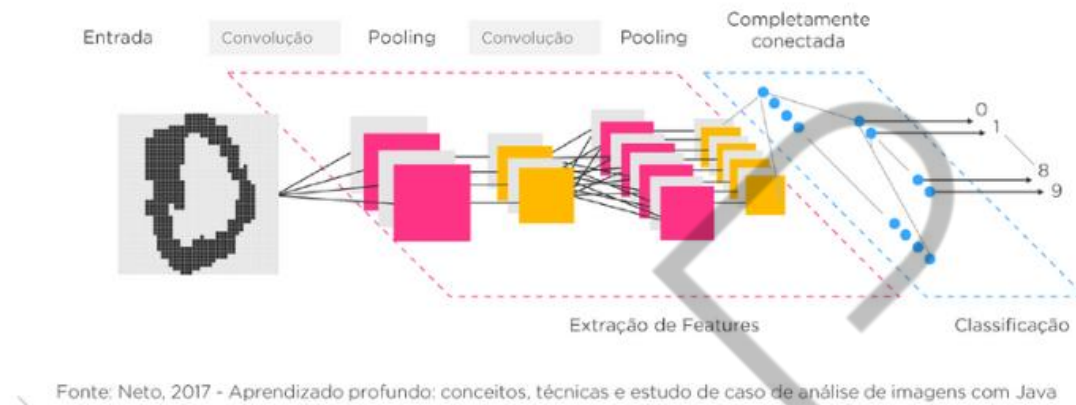


Figura 29 - Arquitetura de redes neurais convolucionais

Fonte: Aprendizado profundo: conceitos, técnicas e estudo de caso de análise de imagens em Java [s.d.]

Observe na imagem que temos a primeira camada chamada de **convolução**, basicamente essa camada é responsável por fazer o papel de "**detector de atributos**". Essa etapa vai mapear a região de uma imagem para posteriormente termos um "**mapa de atributos**". Funciona basicamente como um filtro, na qual chamamos de **kernel** na rede. O filtro nada mais é do que a multiplicação de matrizes, e esse processo é realizado diversas vezes por toda a imagem recebida.

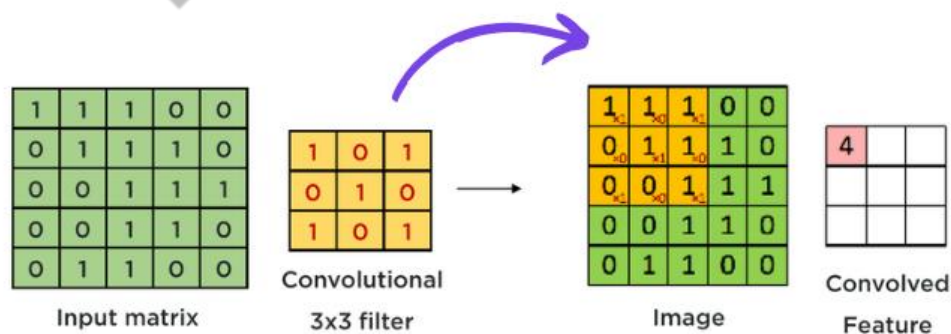


Figura 30 - Camadas convolucionais

Fonte: elaborado pela autora (2024)

Ao realizar a convolução, o mapa de atributos começa a aprender as partes importantes para identificar a composição da imagem (assim como David H. Hubel e Torten Wiesel concluíram que os neurônios do córtex visual de um gato são ativados juntos quando são expostos a algumas linhas ou curvas). Cada pixel será identificado dentro da etapa de convolução, passando pelo filtro kernel movido por um multiplicador, realizando assim a transformação da imagem em um formato compactado.

Essa transformação acontece para a rede mapear quais são os pontos-“chave” que caracterizam essa imagem e armazena em uma representação dimensionalizada para passar para a próxima etapa de convolução da rede. O multiplicador dessa etapa de convolução se chama "**stride (deslocamento)**" (normalmente com o valor = 1) cuja principal função é realizar a **transformação** da imagem. Toda essa transformação resultará em uma imagem com uma dimensão menor.

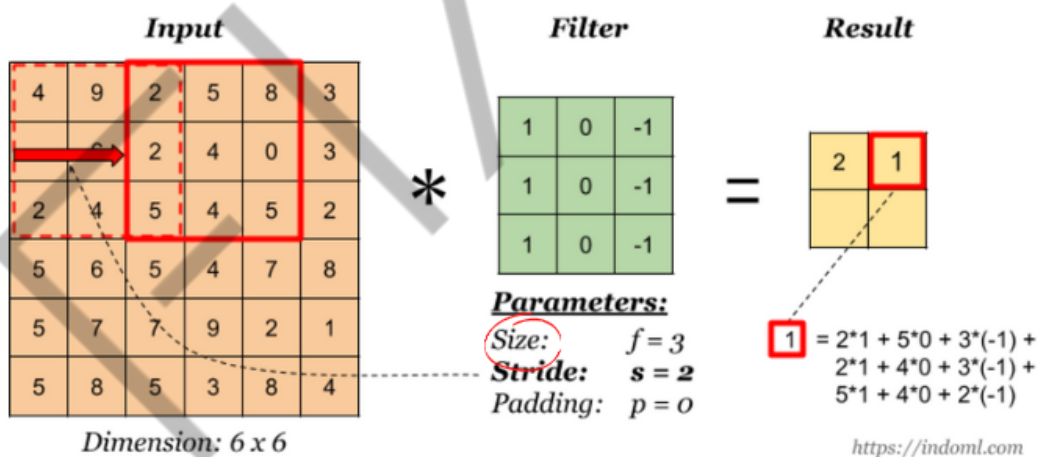


Figura 31 - Deslocamento com stride nas imagens

Fonte: <https://indoml.com> (2022)

É comum adicionar zeros ao redor das imagens de entrada para que uma camada tenha a mesma altura e largura da camada anterior. Esse comportamento é chamado de **padding** na rede. O objetivo do padding é manter a dimensionalidade da imagem. Na configuração das redes neurais convolucionais podemos parametrizar o padding como "**same**" ou "**valid**".

- **Valid** significa que a rede não criará a borda de zeros na imagem e pode ignorar algumas linhas e colunas na parte inferior e direita da imagem de entrada.
- **Same** significa que será criada a borda de zeros na imagem.

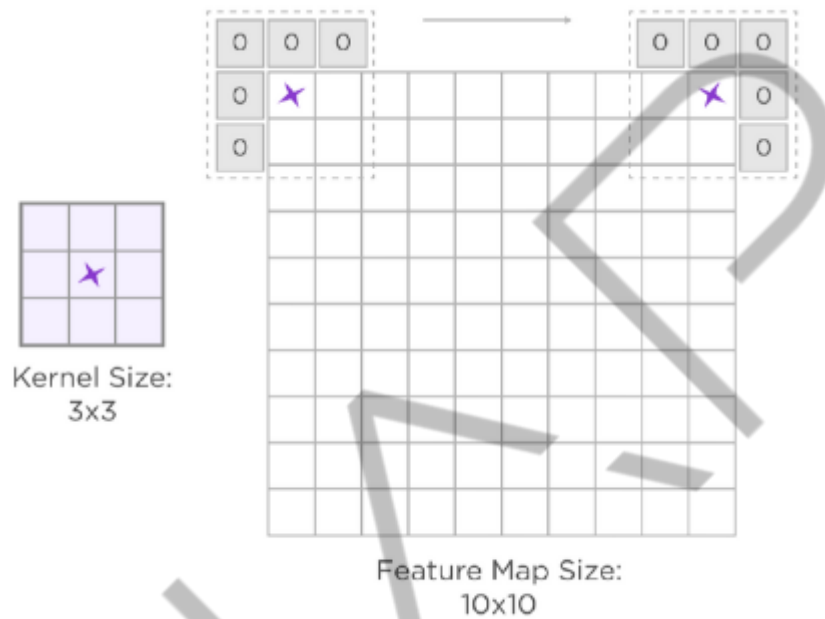


Figura 32 - Padding
Fonte: elaborado pela autora (2024)

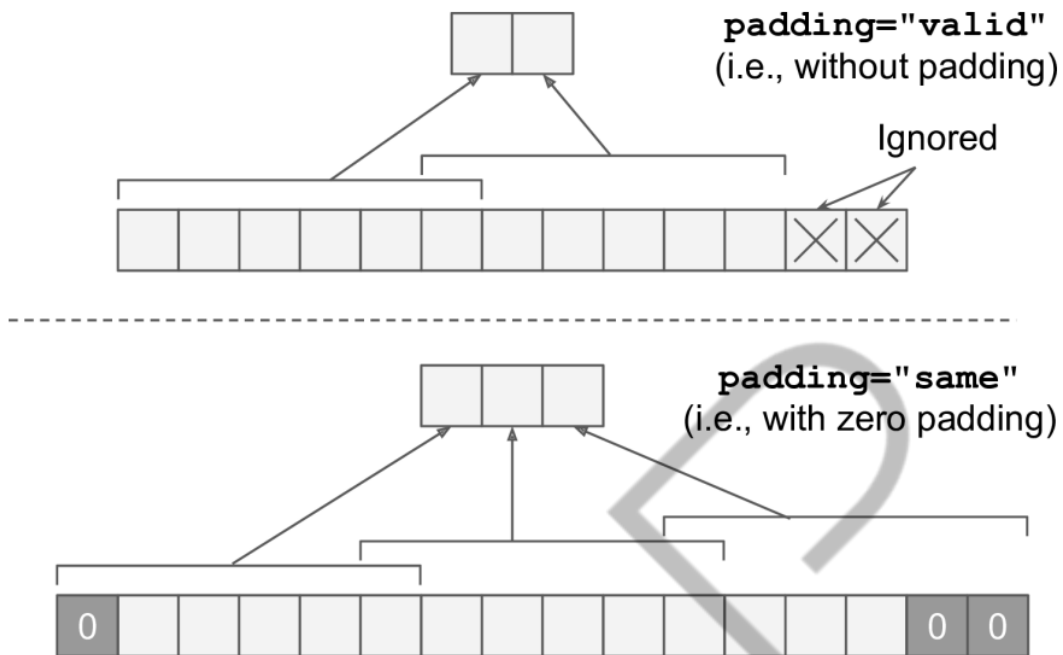
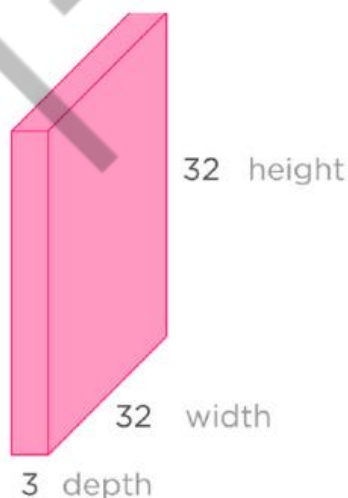


Figura 33 - Padding

Fonte: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow [s.d.]

Imagine que aquela imagem do dígito zero entre na primeira camada convolucional com um tamanho de 32x32x3, ou seja altura e largura é igual a 32 e o 3 significa a escala de cores da imagem, que no caso está em grayscale. Então temos pixels que variam de 0 a 255.



- Imagem 32x32x3 --> Mantém a estrutura espacial preservada

Figura 34 - Exemplo da camada convolucional

Fonte: elaborado pela autora (2024)

Logo em seguida essa imagem passa pelo kernel (filtro), e convolve a imagem, tornando-a em uma dimensão menor de 5X5X3.



Figura 35 - Exemplo da camada convolucional
Fonte: elaborado pela autora (2024)

E por que esses filtros dão certo? Bem, os pesos dos neurônios podem ser representados por uma imagem menor. Durante o treinamento, a CNN encontra os filtros mais úteis para sua tarefa de aprender a combiná-los em padrões mais complexos.

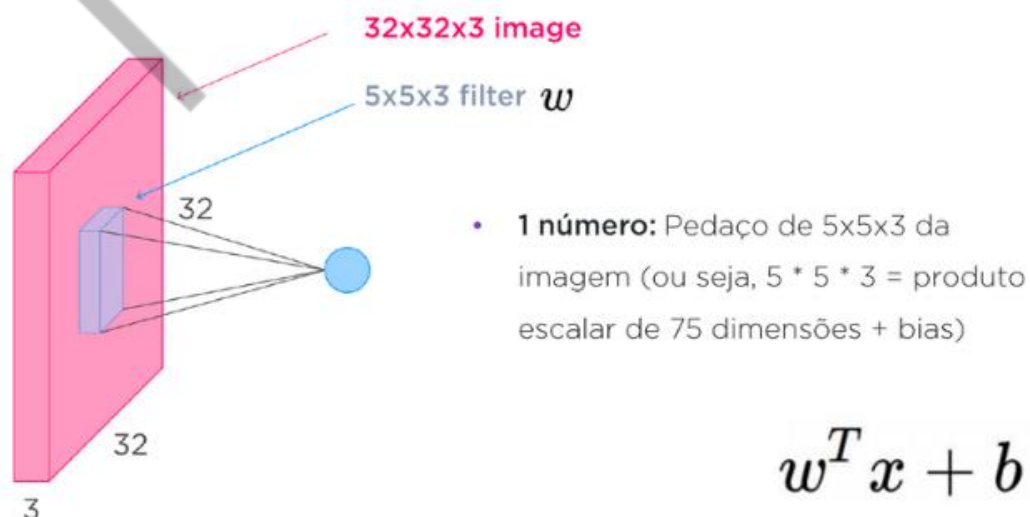


Figura 36 - Exemplo da camada convolucional com kernel
Fonte: elaborado pela autora (2024)

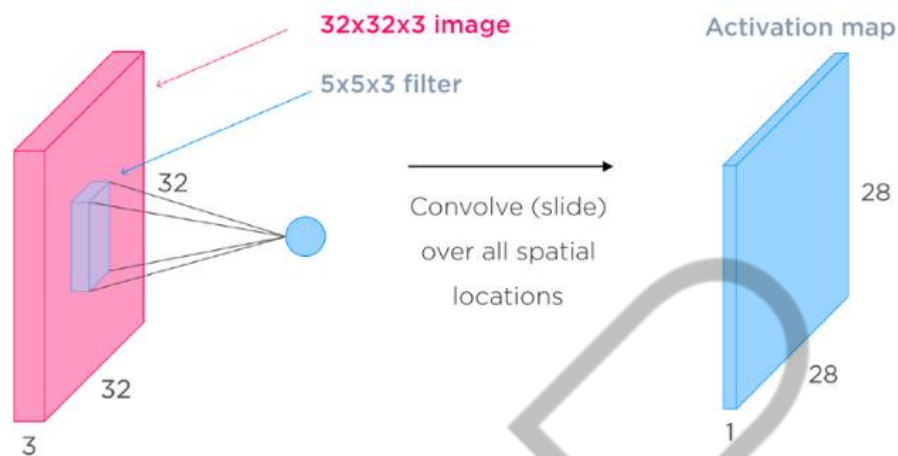


Figura 37 - Mapa de ativação criado na etapa de filtro do kernel
Fonte: elaborado pela autora (2024)

A cada medida que realizamos essas extrações de características das imagens, vamos adicionando novos mapas de filtros. Exemplo: 6 filtros de 5x5 teríamos 6 mapas de ativação.

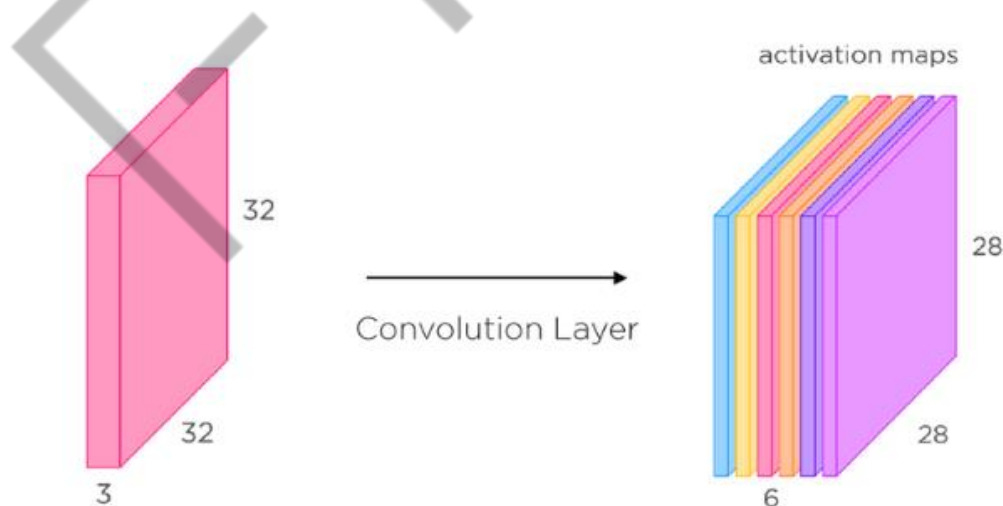


Figura 38 - Mapas de ativação
Fonte: elaborado pela autora (2024)

Entre as camadas de mapa de atributos, teremos uma **função de ativação** para realizar a classificação das imagens. Então nossa rede se torna uma sequência de camadas de convolução, intercaladas por funções de ativação. A função de ativação ReLU é muito utilizada para treinar imagens devido ao seu comportamento de não ativar todos os neurônios ao mesmo tempo (isso traz mais performance para a rede).

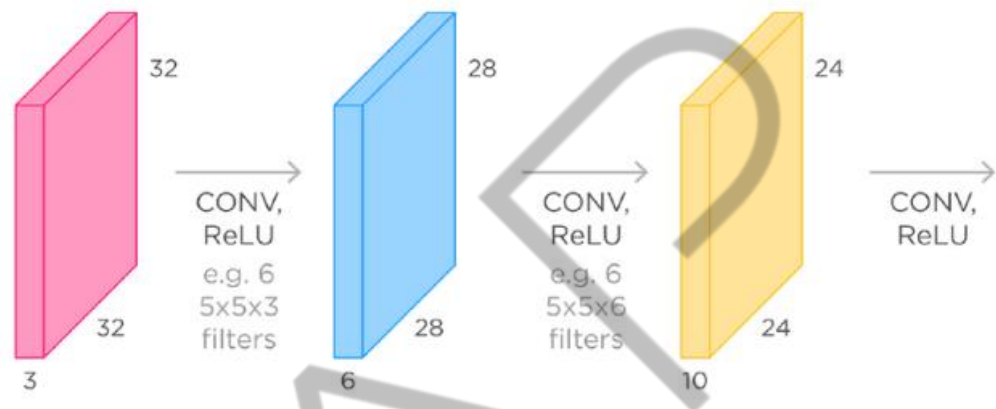


Figura 39 - Mapas de ativação entre funções de ativação
Fonte: elaborado pela autora (2024)

Em seguida, temos a camada de **pooling**. Seu objetivo é subamostrar (ou seja, diminuir) a imagem de entrada para reduzir a carga computacional, o uso da memória e o número de parâmetros (limitando assim o risco de overfitting). A operação **max pooling** retira o maior elemento de determinada região da matriz (imagem). Em seguida é realizado um novo cálculo de stride (assim como na convolução). A principal motivação dessa operação no modelo, é de diminuir sua variância a pequenas alterações e também de reduzir a quantidade de parâmetros treinados pela rede.

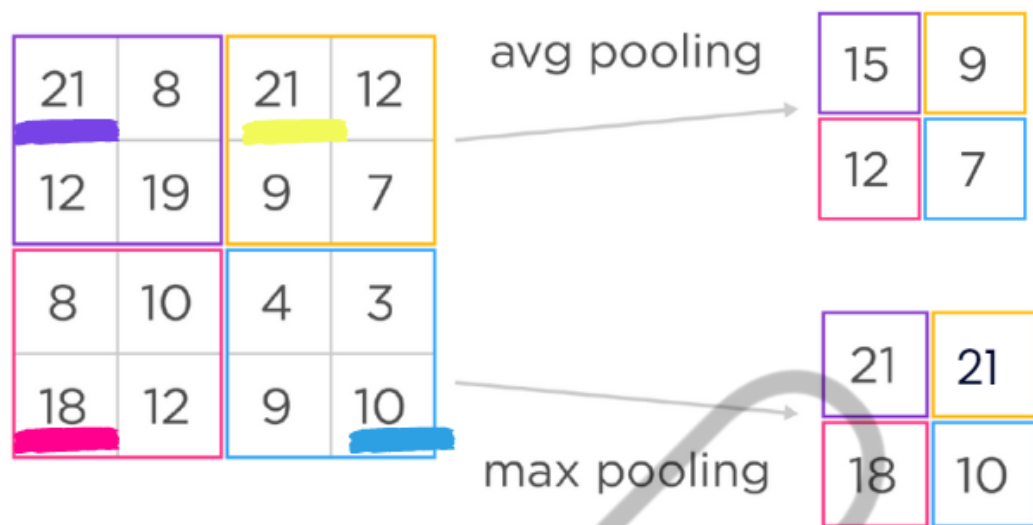


Figura 40 - Max e AVG Polling
 Fonte: elaborado pela autora (2024)

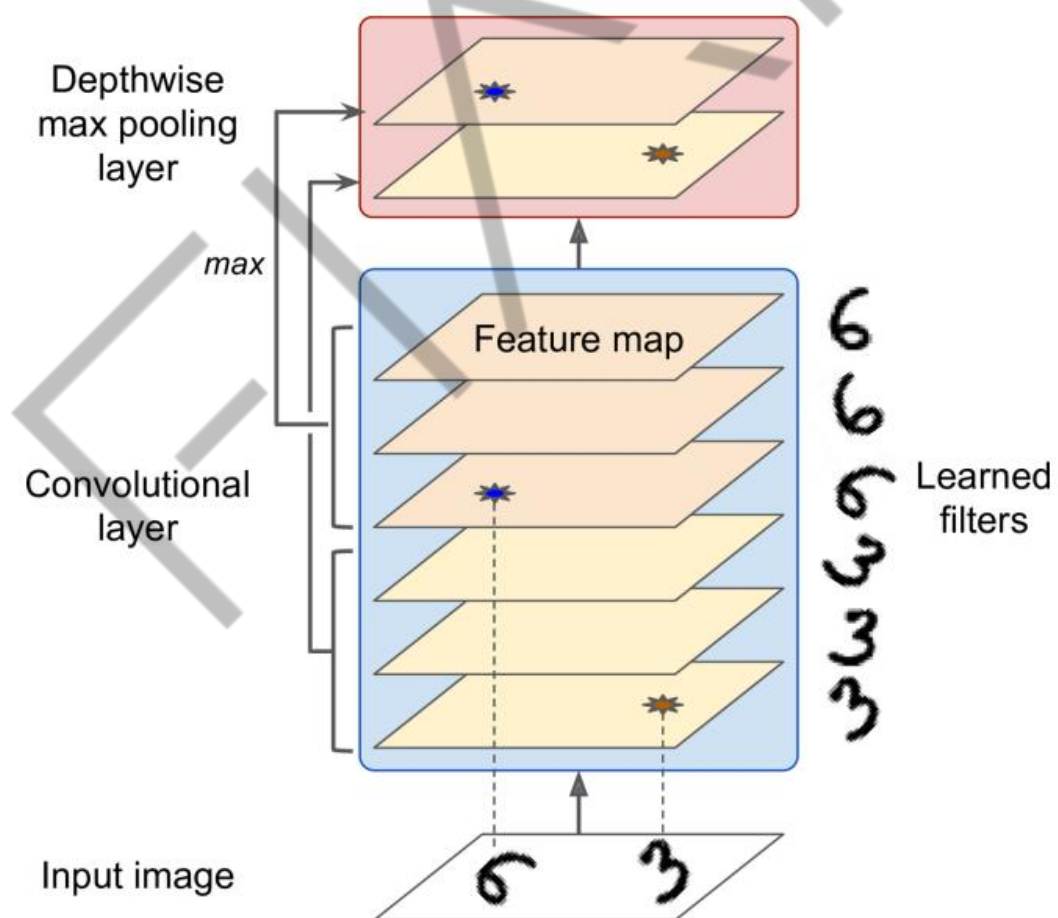


Figura 41 - O pooling máximo em profundidade pode ajudar a CNN a aprender qualquer invariância
 Fonte: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow [s.d]

Conhecendo sobre as RRNs (Redes Neurais Recorrentes)

Podemos definir as redes neurais recorrentes como uma rede que trabalha com **sequências de comprimentos arbitrários, ou seja, dados que dependem de sequências anteriores à sua tarefa atual**, tal como as séries temporais ou textos que dependem de palavras anteriores para fazer sentido (trazer o contexto). Uma rede neural recorrente é muito parecida com as redes neurais feedforward, exceto pelo fato de obter conexões de neurônios que apontam para trás. Podemos dizer que essa rede é nostálgica! Ela trabalha com a noção de tempo a todo o momento para aprender a usar informações passadas ao seu favor. Rede inteligente essa, não acha?

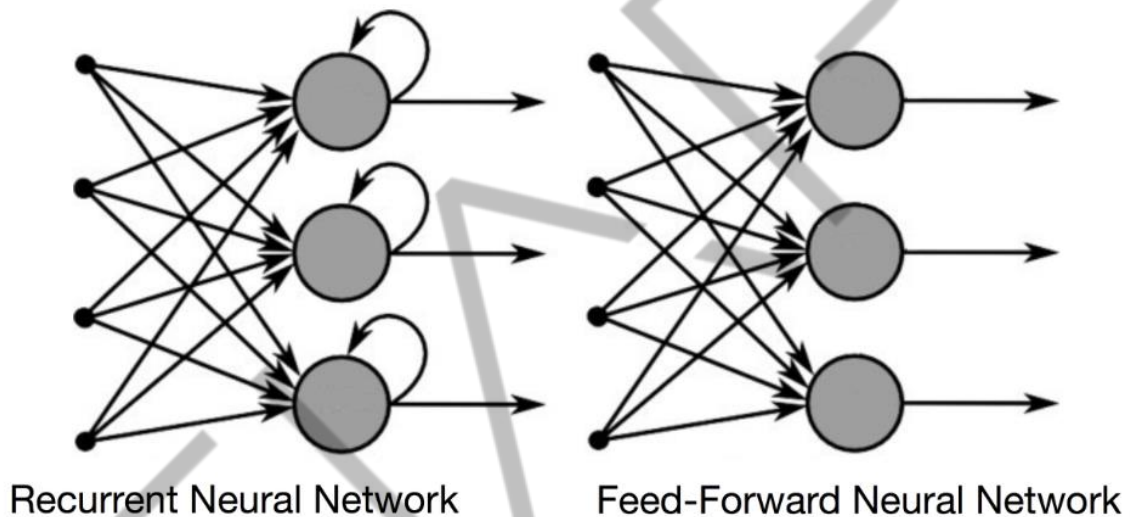


Figura 42 - Diferença entre uma rede Feed-Forward e uma Rede Neural Recorrente
Fonte: <https://www.deeplearningbook.com.br/redes-neurais-recorrentes> (2022)

Podemos definir que a rede neural recorrente **aprende com ela mesma** (assim como nós humanos aprendemos com nossos erros). Basicamente, esse tipo de arquitetura aprende não só com os dados de entrada, mas também com as próprias saídas da rede (muito parecido com um looping de aprendizado, por isso chamamos de redes recorrentes). A rede vai trabalhar com os dados de entrada e seus respectivos outputs em um certo estado t de tempo que a rede está aprendendo. Observe a equação abaixo, que explica o funcionamento das redes neurais recorrentes:

$$\mathbf{h}_t = \phi(W\mathbf{x}_t + U\mathbf{h}_{t-1}),$$

Figura 43 - Equação de RNN

Fonte: <https://www.deeplearningbook.com.br/redes-neurais-recorrentes> (2022)

A equação nos mostra:

- h_t : estado atual da célula de memória.
- ϕ : função de ativação (pode ser uma sigmóide ou tangente hiperbólica).
- w : matriz de pesos.
- x_t : input atual.
- U : estado anterior (matriz oculta de pesos).
- h_{t-1} : estado passado.

Podemos concluir então que h_t é dado sobre a função da entrada na mesma etapa de tempo x_t (estado atual da célula), modificada por uma matriz de peso W (como a que usamos para redes feedforward) adicionada ao estado oculto do passo de tempo anterior h_{t-1} (estado passado) multiplicado por seu próprio estado oculto – para a matriz de estado oculto U (cadeia de Markov). Basicamente, o estado atual da célula de memória trabalha com os inputs atuais, mas também depende dos inputs do estado anterior de memória. Por isso essa rede trabalha com dados do passado, ela é nostálgica!

Para ficar mais claro o entendimento, vamos pensar no seguinte exemplo aplicado em processamento de linguagem natural: imagine um modelo de RNN que tenta prever a próxima palavra baseada nas anteriores, assim como o gmail e o outlook possuem essa inteligência de autocompletar palavras.

Segue documento em anexo.

Figura 44 - Exemplo de RNN
Fonte: elaborado pela utora (2024)

É bem óbvio que a última palavra será “anexo”, certo? Em tais casos, onde o gap entre a informação relevante e o lugar que ela é requisitada é pequeno, as RNNs podem aprender a usar informações passadas. Repare aqui que o contexto se refere apenas a termos que estão antecedentes, afinal, para a rede prever corretamente a palavra “anexo”, precisamos saber que ela está antecedita das palavras “segue” e “documento”. Porém, imagine como a rede preveria a seguinte situação:

O tempo que passei na China foi muito legal e tive a oportunidade de aprender a falar chinês, um idioma incrível.

Figura 45 - Distância do gap de informação para compreender contexto
Fonte: elaborado pela autora (2024)

O gap entre a informação relevante (início da frase) e o ponto onde ela é requisitada (fim da frase, a palavra “chinês”) pode se tornar bem grande. Infelizmente, conforme esse gap aumenta de tamanho, as redes neurais recorrentes comuns se tornam incapazes de aprender a conectar as informações. O que acontece na verdade é que elas sofrem do chamado **vanish gradient problem**.

Imagine uma situação em que estamos analisando o contexto de uma frase apenas levando em consideração as palavras do início? Muitos comentários em pesquisas de satisfação, por exemplo, podem começar com elogios, mas logo em seguida ser composto por algum comentário negativo ou com algum ponto de atenção. Isso poderia enviesar nossa classificação de sentimentos, certo?

Gostei muito do filme novo da Marvel! Foram muito fieis aos uniformes iguais à dos quadrinhos, **porém poderiam ter focado mais** no início da história do Peter Parker, ficou meio sem sentido o filme.

Figura 46 - Exemplo mais complexo de uso da RNN
Fonte: elaborado pela autora (2024)

O **vanishing gradient problem** ocorre quando o **gradiente desaparece conforme ocorre o backpropagation**. É isso que leva ao “esquecimento” das informações mais antigas devido às transformações pelas quais os dados passam ao atravessar uma RNN. Para solucionar esse tipo de deficiência das RNNs tradicionais, temos redes recorrentes mais evoluídas, tais como a redes neurais **LSTM Long Short-Term Memory** (memória de curto e longo prazo) e os modelos **Transformers**. Como o grande foco dessa disciplina é a IA generativa, vamos focar nos fundamentos dos modelos transformers. Além de todo o comportamento aprendido até agora com as deep learnings, desde as funções de custo até o que é o gradiente descendente, agora chegou o momento de aprendermos os fundamentos do processamento de linguagem natural que são utilizados nos modelos de IA generativas: os famosos embeddings.

Entendendo os embeddings

Os embeddings de texto são uma técnica dentro do mundo de processamento natural de texto (NLP) responsável por **converter os textos (tokens) em vetores numéricos**. O grande diferencial dos embeddings das representações vetoriais de palavras em números, tal como BOW (Bag of word) e TF-IDF (Term Frequency-Inverse Document Frequency), é que **os embeddings realizam a captação semântica de textos e contextos**, a partir de tokens mais próximos dentro do embedding treinado.

As técnicas tradicionais aprendidas nas aulas de NLP são boas para determinados tipos de tarefas, mas elas podem ser falhas quando estamos falando de um **grande volume de dados**. Imagina criar uma matriz BOW para todo o conteúdo da internet? Bem, criaríamos uma matriz esparsa e não seria eficiente por

conta da falta de contexto entre os tokens. Os modelos de IA generativa, tal como os LLMs (Large Language Models), necessitam dos embeddings para gerar conteúdos eficientes e com sentido cognitivo.

Existem muitas formas de realizar a tokenização das palavras (representação numérica), uma forma mais simples de realizar a tokenização é **mapeando cada palavra do nosso corpus de documento**, sendo uma palavra **representada por um número que pode ser interpretado por um índice da palavra**. Vamos pensar no seguinte exemplo:

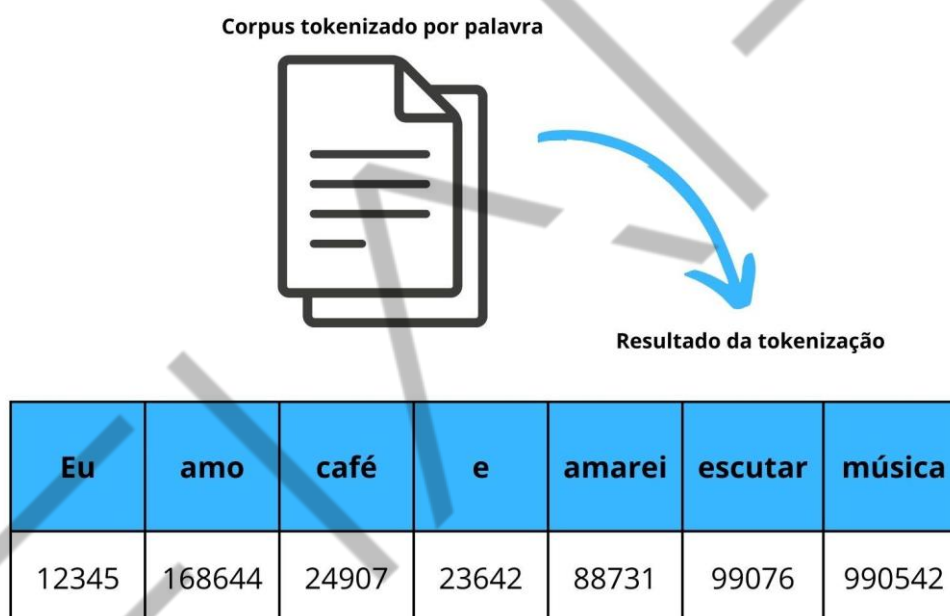


Figura 47 - Exemplo de tokenização por palavra
Fonte: elaborado pela autora (2024)

Nesse caso temos nessa frase um total de **7 tokens**. Essa estratégia com certeza é muito mais eficiente do que a estratégia de tokenização por cada palavra do alfabeto, mas em contrapartida, aumenta a complexidade do dicionário. Imagine o tanto de palavras que seria necessário mapear para tentar criar um dicionário de tokens eficiente? Outras técnicas são mais interessantes, como por exemplo as técnicas que já utilizamos no clássico NLP, como a **Stemização** e a **Lematização**.

Stemização: imagine um explorador desbravando uma floresta densa. A stemização é como um machete afiado, cortando galhos e folhas (afixos) para chegar ao tronco principal (radical) da palavra de forma rápida e eficiente. Ela **remove**

sufixos e prefixos comuns, mas nem sempre se preocupa com a precisão gramatical. Por exemplo:

- "Andando", "andamos", "andaram" se transformam em "and"
- "Carro", "carrinhos", "carregando" viram "carr"

Lematização: agora, imagine um estudioso meticuloso mapeando cada árvore da floresta. A lematização é como um mapa detalhado, guiando-nos à forma canônica da palavra, levando em consideração o contexto e a gramática. Ela consulta dicionários e regras linguísticas para encontrar a forma correta, mesmo que seja menos comum no dia a dia. Por exemplo:

- "Andando", "andamos", "andaram" se transformam em "andar"
- "Carro", "carrinhos", "carregando" viram "carro"

Se aplicarmos uma tokenização com lematização, por exemplo, diminuiremos o número dos tokens do exemplo acima para 6 tokens.

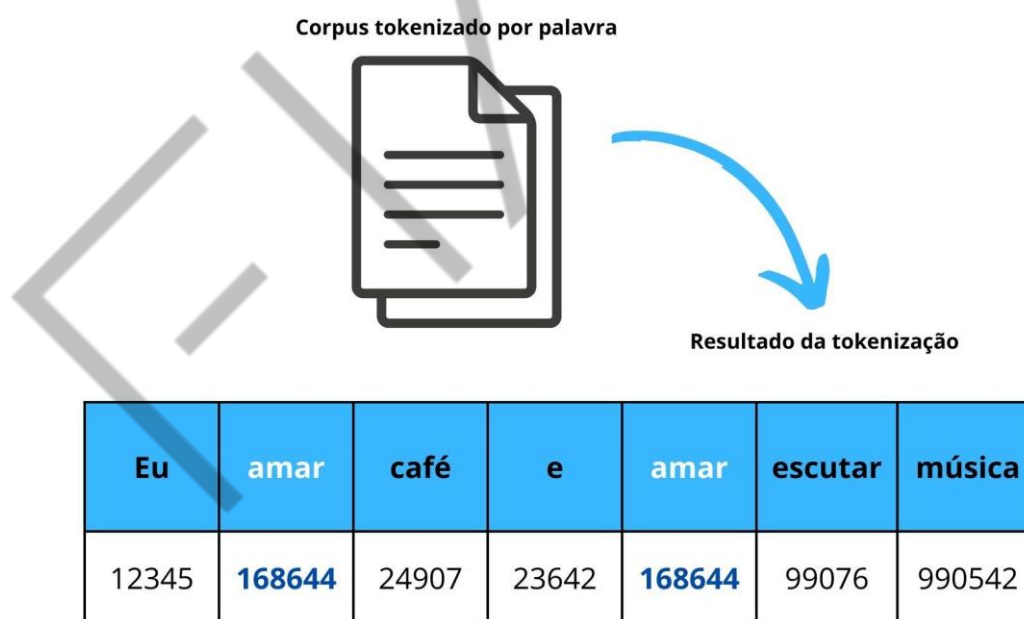


Figura 48 - Exemplo de tokenização por palavra
Fonte: elaborado pela autora (2024)

Caso você queira explorar o processo de 8k de uma IA generativa bem famosa, eu recomendo testar no ["tokenização da OpenIA"](#):

Learn about language model tokenization

OpenAI's large language models (sometimes referred to as GPT's) process text using **tokens**, which are common sequences of characters found in a set of text. The models learn to understand the statistical relationships between these tokens, and excel at producing the next token in a sequence of tokens.

You can use the tool below to understand how a piece of text might be tokenized by a language model, and the total count of tokens in that piece of text.

It's important to note that the exact tokenization process varies between models. Newer models like GPT-3.5 and GPT-4 use a different tokenizer than previous models, and will produce different tokens for the same input text.

GPT-4o (coming soon)
GPT-3.5 & GPT-4
GPT-3 (Legacy)

Eu amo café e amarei escutar música

Clear
Show example

Tokens	Characters
10	35

Eu amo café e amarei escutar música

Figura 49 - Exemplo de tokenização na OpenIA (Modelo Chat GPT 3.5 e 4)
Fonte: elaborado pela autora (2024)

Bem, agora que entendemos os tokens, vamos entender os embeddings? Enquanto o token é uma representação estática o embedding é mais dinâmico! Como eu tinha dito no início da apresentação dos embeddings, realizamos uma representação vetorial das palavras, mas em um **espaço multidimensional**. Basicamente no plano multidimensional dos embeddings, **as palavras com contexto semântico parecido estarão próximas umas das outras por conta da similaridade**. Pense que cada palavra será representada por uma coordenada. Então se tivermos uma palavra “café”, a palavra “leite” estaria muito mais próxima dela do que a palavra “cachorro”.

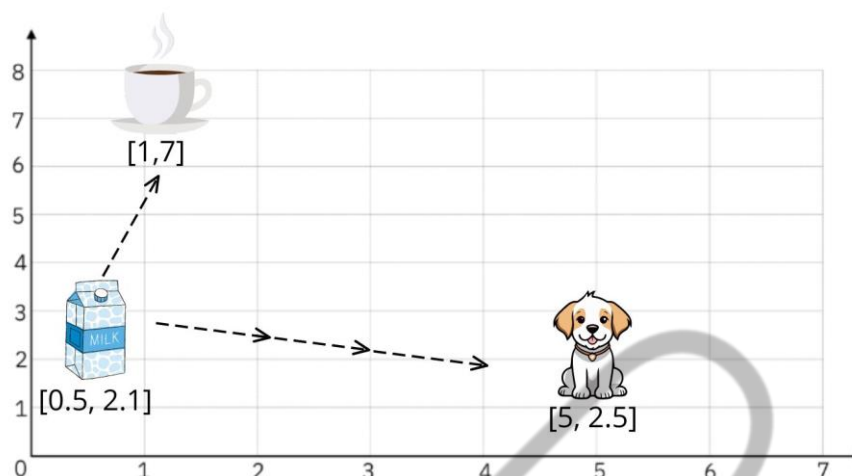


Figura 50 - Exemplo de posição de tokens no embeddings
 Fonte: elaborado pela autora (2024)

Importante ressaltar que na realidade, mapeamos embeddings em muito mais do que apenas 3 dimensões. Na prática trabalhamos com embeddings em torno de 512 a 4096 dimensões, mas para fins didáticos vale a pena imaginarmos dessa forma. Pensarmos em palavras similares é muito simples, mas e o que acontece com palavras que possuem duplo sentido?

Acompanhe o exemplo a seguir para compreender o funcionamento dos embeddings com o mecanismo de atenção. Imagine que um modelo generativo de LLM esteja recebendo a seguinte pergunta: “Quais são os dados necessários para realizar uma viagem internacional?” O significado de dados pode estar associado com muitos contextos, não é mesmo? Podemos associar com informações com dados de tabuleiros, por exemplo.

Com base no contexto de palavras que são utilizadas no prompt, essas palavras são transformadas e mapeadas pelo embeddings utilizando produto escalar, e o mecanismo de atenção levará em consideração quais são as palavras de maior atenção com base no cálculo de distância mais curta entre as palavras, encontrando assim qual é o melhor contexto para retornar um novo conteúdo como resposta.

Quais são os dados necessários para realizar uma viagem internacional?

Para realizar uma viagem internacional, é necessário reunir diversos documentos e informações. Aqui estão os dados e documentos essenciais que você precisará:

Documentos Pessoais

1. Passaporte:

- Deve estar válido por pelo menos seis meses após a data de retorno prevista.
- Verifique se há páginas suficientes para carimbos de entrada e saída.

2. Visto (se aplicável):

- Verifique se o país de destino exige visto para cidadãos do seu país.
- Certifique-se de solicitar o visto com antecedência e cumprir todos os requisitos.

Mensagem ChatGPT

ChatGPT pode cometer erros. Considere verificar informações importantes.

Figura 51 - O sentido do contexto importa para os conteúdos gerados pelas LLMs
Fonte: elaborado pela autora (2024)

Vamos analisar qual é o resultado com a palavra **“dados”** sobre um novo contexto? Que tal **“Quais são os dados utilizados no jogo de cartas?”**

Quais são os dados utilizados no jogo de cartas?

Os jogos de cartas utilizam diversos dados e elementos que variam conforme o tipo de jogo. Aqui estão os dados comuns utilizados na maioria dos jogos de cartas:

Componentes Essenciais

1. Baralho de Cartas:

- **Cartas:** O número de cartas e o tipo de baralho podem variar (baralho francês padrão com 52 cartas, baralho espanhol, baralho de tarô, etc.).
- **Naipes:** As cartas são divididas em naipes (copas, ouros, espadas e paus no baralho francês).
- **Valores:** As cartas têm valores numéricos ou figuras (ás, 2-10, valete, dama e rei).

Dados do Jogo

1. Número de Jogadores:

Mensagem ChatGPT

ChatGPT pode cometer erros. Considere verificar informações importantes.

Figura 52 - O sentido do contexto importa para os conteúdos gerados pelas LLMs
Fonte: elaborado pela autora (2024)

O QUE VOCÊ VIU NESTA AULA?

Esta aula foi muito importante para você compreender o funcionamento das IA generativas. Sendo assim, você aprendeu quais são as principais bases dos modelos generativos e agora fica mais fácil compreender toda a arquitetura desses modelos robustos!

EMENDAS

REFERÊNCIAS

DEEP Learning Book. Disponível em: <https://www.deeplearningbook.com.br/>. Acesso em: 25 jun. 2024.

GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. 2ª Edição. Califórnia: O'Reilly Media, Inc., 2019.

GUIA de embeddings. **Google AI for Developers**. Disponível em: <https://ai.google.dev/gemini-api/docs/embeddings?hl=pt-br>. Acesso em: 25 jun. 2024.

LOPES, A. Token e Embedding: conceitos da IA e LLMs. **Brains.dev**. Disponível em: <https://brains.dev/2024/token-e-embedding-conceitos-da-ia-e-llms/>. Acesso em: 25 jun. 2024.

PALAVRAS-CHAVE

Deep Learning. Embeddings. Funções de ativações. Tokens. Backpropagation.

EMEND



POSTECH